

Documentación Técnica

JULIO 2026

Índice.

Servicios Externos de los Snapshots.	2
01 Obtener Canales y Camiones.	4
Endpoint.	5
Especificación de DTOs y funciones.	5
02 Crear Plan de Despacho (DRAFT)	10
3. Especificación de DTOs y funciones.	11
03 Seleccionar Filtro.	13
Endpoint.	14
Especificación de DTOs y funciones.	14
04 Crear Órdenes de Planificación.	17
Especificación de DTOs y funciones.	18
05 Obtener Traslados	22
Endpoint.	23
3. Especificación de DTOs y funciones.	23
06 Obtener Devoluciones.	25
Especificación de DTOs y funciones.	26
07 Agregar Devoluciones y Transferencias a la Planificación	28
Endpoint.	29
Especificación de DTOs y funciones.	29
Sub-DTO: TransferCandidateDto (Traslados).	29
Sub-DTO: ReturnCandidateDto (Devoluciones/Recojos).	30
B. createCandidateOrder() 7.3	33
08 Obtener Puntos de entregas.	35
Especificación de DTOs y funciones.	36
09 Optimización Multiple de Rutas (Google Route Optimization)	39
Enpoint.	40
Especificación de DTOs y Funciones	40
10 Crear Rutas	54
Enpoint.	55
11 Obtener Ordenes de transporte	62
12 Listar puntos de entregar para unificar ordenes de transporte.	65
Enpoint.	65

13 Mover puntos de entrega entre rutas.....	67
14 Procesar Orden de Transporte	70
Tabla Descriptiva del Retorno (transportOrderData)	72
Tabla Descriptiva de Entrada (transportOrderSapDto)	75
Tabla Descriptiva de Salida (sapResponse)	76
Tabla Descriptiva de Parámetros -> Tabla transport_orders	77
JSON de Respuesta Final al Frontend	77
15 Optimización de Rutas para ordenes de transporte Unificadas.....	77
Enpoint.	78
Especificación de DTOs y Funciones	78
16 CREACION DE ORDENES DE TRANSPORTE FINALIZADA.....	86
Tabla Descriptiva de Entrada (unifyTransportOrderDto).....	87
JSON.	88
Tabla Descriptiva (transportOrderDto) -> Tabla transport_orders y transport_order_sales	90
Tabla Descriptiva (transportOrderSapDto) -> Servicio zws_orden_transporte_crear.....	91
17 Obtener Camiones y rutas	92
17.1 Request (Frontend → Gateway Controller → Planning Controller).....	93
17.3 transport_orders - findByDistributorId(distributorId).....	94
17.5 routes - findByTripId(trip.id, { orderStatus })	94
19 Obtener ordenes Despachadas	96
24 Tracking del camión	106
18 Revision a Ciegas.....	106
Tabla Descriptiva del Query Parameter (Entrada).....	107
JSON de Ejemplo de Entrada (HTTP Query Parameters)	107
Tabla Descriptiva del Retorno (activeTripsList).....	107
JSON de Ejemplo de Salida (activeTripsList)	108
Tabla Descriptiva del Retorno (transportOrdersList)	109
JSON de Ejemplo de Salida (transportOrdersList)	109
JSON de Ejemplo de Salida (stopsCount)	110
Tabla Descriptiva del Retorno Final (blindCountOrdersResponse).....	111
JSON de Ejemplo de Salida Final (blindCountOrdersResponse)	112

21 Obtener detalle de una orden de transporte (conteo a ciegas).	112
Tabla Descriptiva de Entrada (transportOrderId)	113
JSON / Path Parameter de Entrada.	113
Tabla Descriptiva de Entrada	114
JSON de Ejemplo de Entrada	114
Tabla Descriptiva de Salida (transportOrderData)	114
JSON de Ejemplo de Salida (transportOrderHeader)	114
Tabla Descriptiva de Entrada	115
JSON de Ejemplo de Entrada	115
Tabla Descriptiva de Salida (requestedItemsList)	115
JSON de Ejemplo de Salida (requestedItemsList)	115
Tabla Descriptiva de Entrada	116
Tabla Descriptiva de Salida (existingCountsList)	116
JSON de Ejemplo de Salida (existingCountsList)	117
Tabla Descriptiva de Entradas	118
JSON de Ejemplo de Entrada	118
Tabla Descriptiva de Salida (blindCountOrderDetailResponse)	118
JSON de Ejemplo de Salida	119
22 Registro y eliminacion de Items.	119
Tabla Descriptiva de Parámetros de Entrada -> Tabla truck_inventories	125
23 Finalizacion de Revision a ciegas.	126
JSON de Ejemplo de Entrada (finalizeBlindCountDto)	127
JSON de Ejemplo de Entrada (historyDto)	133

Servicios Externos de los Snapshots.

01 Delivery Point

Parametro de entrada: deliveryPointId, ownerId, customerId.

Salida:

```
[
  {
    "deliveryPointId": 45,
    "ownerId": 4,
    "ownerName": "Cliente padre 1",
    "customerId": 78,
    "customerName": "Cliente hijo 2",
    "latitud": -17.26564,
    "longitud": 49.4564
  },
  {...}
]
```

01 Sales Order y Sales Order Item

Parametro de entrada: saleOrderId, deliveryPointId

Salida:

```
[
  {
    "saleOrderId": 21,
    "deliveryPointId": 2,
    "documentId": 1000026565,
    "totalWeightKg": 1656,
    "totalVolumeM3": 3500,
    "item": [
      {
        "productId": 5,
        "quantityMin": 24
      }
    ]
  },
  {...}
]
```

01 Product

Parametro de entrada: productId

Salida:

```
[
  {
    "productId":78,
    "productSKU":"SKU-MR-1",
    "weight":45.66,
    "volume":20,
  },
  {...}
]
```

01 Obtener Canales y Camiones.

Endpoint.

Tipo: (HTTP) GET /planning

Obtener los datos necesarios para realizar la planificacion.

Especificación de DTOs y funciones

Request Principal (GET /planning)

Se requiere el id de la distribuidora del usuario planificador, filtros para obtener los pedidos y los camiones en los parámetros.

Parámetros de entrada: filterTrucksDto

Tabla de atributos filterTrucksDto

Atributo	Tipo	Oblig.	Descripción / Restricción
truckType	string	No	Tipo de vehículo.
status	string	No	Estado del vehículo.
plate	string	No	Placa de vehículo.
vehicleClassification	string	No	Clasificación de vehículo.

Request en JSON.

```
"filterTrucks": {  
  "truckType": "FRI0",  
  "status": "DISPONIBLE",  
  "plate": "2345-XYZ",  
  "vehicleClassification": "PESADO"  
}
```

A. getChannels() 1.1

Función responsable de enviar los datos al microservicio ms_sales para retornar los pedidos agrupados por canal.

- **Parámetro de salida:** listOrders

Tabla de atributos de listOrders

Atributo	Tipo TypeScript	Validación	Oblig.	Descripción / Restricción
channelId	string	@IsString()	Sí	Id del canal
channelName	string	@IsString()	Sí	Nombre del canal
countOrders	number	@IsInt()	Sí	Cantidad de pedidos agrupados
countCustomers	number	@IsInt()	Sí	Cantidad de clientes agrupados
total	number	@IsNumber()	Sí	Sumatoria total monetaria (ej: 5200.99)
totalWeight	number	@IsNumber()	Sí	Sumatoria total kg (ej: 750.55)

B. Lógica de Obtención de Flota (getTrucks 1.2 y getTrucksSap 1.2)

El sistema implementa una validación condicional para optimizar las consultas, priorizando la base de datos local (truck DB).

1. **Si existe en DB (getTrucks(filterTrucksDto) 1.2):** El Planning Service consulta la tabla local de camiones (truck_master) aplicando los filtros. Si se encuentran registros actualizados, se retorna esta lista de camiones directamente, omitiendo la llamada a SAP.
2. **Si no existe en DB (getTrucksSap(filterTrucksDto) 1.2):** Si la consulta local no devuelve resultados (o la caché expiró), se realiza la petición al sistema SAP a través de la función zws_camion_listar.
3. **saveTrucks(truckDto):** Una vez que SAP retorna la lista de camiones, el servicio utiliza TypeOrm (mediante un bulkCreate o upsert) para persistir esta nueva información en la base de datos local (truck DB), manteniéndola sincronizada para futuras consultas.

Tabla DTO: TrucksResponseDto (Respuesta de SAP)

Atributo	Tipo	Oblig.	Descripción / Mapeo
camion_id	string	Sí	Número de equipo SAP (ID del vehículo).
status	string	Sí	Estado operativo actual.
license_plate	string	Sí	Placa del vehículo.
description	string	Sí	Descripción técnica o comercial.
default_driver	string	No	Chofer predeterminado asignado.
vehicle_type	string	Sí	Tipo de vehículo (Ej: CAMION, FURGON).
is_refrigerated	boolean	Sí	Bandera si cuenta con equipo de frío.
max_weight	number	Sí	Capacidad máxima de carga (peso).
weight_unit	string	Sí	Unidad de medida de peso (Ej: KG).
max_volume	number	Sí	Capacidad máxima de carga (volumen).
volume_unit	string	Sí	Unidad de medida de volumen (Ej: M3).
property	string	No	Propiedad (ej. Propio, Tercerizado).
vehicle_class	string	Sí	Clase de Vehículo.
distributor	string	Sí	Centro o Distribuidora asignada.
shift_start	string	Sí	Inicio del turno.
shift_end	string	Sí	Fin del turno.

C. calculateProfit() 1.2a

Función responsable de calcular la utilidad de cada camión. Para ello, se realiza una consulta cruzada a la tabla truck_inventory de la base de datos local usando los IDs de los camiones obtenidos en el paso B.

- **Parámetro de entrada:** listTrucksId (Arreglo de IDs)
- **Parámetro de salida:** TrucksResponseDto (Se le inyecta la propiedad profit)

Ejemplo JSON (Respuesta Final de Camiones con Profit)

JSON

```
{
  "success": true,
  "code": 200,
  "data": [
    {
      "camionId": 880012,
      "status": "ACTIVO",
      "licensePlate": "3456-ABC",
      "description": "Furgón Nissan Condor 5T",
      "defaultDriver": "Carlos Mamani",
      "vehicleType": "FURGON",
      "isRefrigerated": false,
      "maxWeight": 5000.00,
      "weightUnit": "KG",
      "maxVolume": 18.50,
      "volumeUnit": "M3",
      "property": "PROPIO",
      "vehicleClass": "Camion",
      "distributor": "GV05",
      "profit": 0.75,
      "shiftStart": "08:00",
      "shiftEnd": "17:00"
    }
  ]
}
```

D getCities() 1.3

Función que obtiene la lista de vendedores por distributorId.

Parámetro de entrada: **distributorId**.

```
{ "distributorId": 6 }
```

Parámetro de salida:

```
[
  {
    "id": 3,
    "name": "Santa Cruz",
  },
  {
    "id": 4,
    "name": "Montero",
  }
]
```

QueryParamsDto.

Dto para filtrar la informacion que tiene como atributos distributorId y cityId.

Json.

```
{ "cityId": 6 }
```

E getEmployee() 1.4.

Funcion que obtiene la lista de empleados.

Parámetro de entrada: **queryParams**.

Parámetro de salida.

```
[
  {
    "employeeId": 1,
    "name": "D. Cespedes" // add distributorId, code, rol
  },
  {
    "employeeId": 2,
    "name": "J. Rojas"
  }
]
```

F getMarkets() 1.5.

Funcion que obtiene la lista de mercados.

Parámetro de entrada: **queryParams**.

Parámetro de salida.

```
[
  {
    "marketId": 7,
    "name": "Norte"
  },
  {
    "marketId": 8,
    "name": "Abasto"
  }
]
```

G getZones() 1.6.

Funcion que obtiene la lista de zonas.

Parámetro de entrada: **queryParams**.

Parámetro de salida.

```
[
  {
    "zoneId": 4,
    "name": "Zona Norte"
  },
  {
    "zoneId": 5,
    "name": "Zona Sur"
  }
]
```

02 Crear Plan de Despacho (DRAFT)

Enpoint.

Tipo: (HTTP) POST /dispatch-plan

3. Especificación de DTOs y funciones.

Request Principal (POST /dispatch-plan)

Se requiere la información general de la planificación y el listado de camiones con sus respectivos turnos y capacidades.

Parámetros de entrada: CreateDispatchPlanDto.

Atributo	Tipo	Oblig.	Descripción / Restricción
planStatusId	number	Sí	FK de la tabla plan_status
distributorId	string	Sí	ID del almacén/distribuidora origen
planDate	Date	Sí	Fecha programada para la ejecución del plan
plannedOrderCount	number	No	Cantidad de pedidos iniciales contemplados (puede ser 0 si solo se apartan camiones)
plannedTruckCount	number	Sí	Cantidad de camiones planificados (debe coincidir con la longitud del array trucks)
createdBy	string	Sí	Identificador del empleado/planificador
trucks	TruckPlanDto[]	Sí	Arreglo de camiones asignados al plan

Tabla de atributos TruckPlanDto (Sub-DTO para los camiones)

Atributo	Tipo	Oblig.	Descripción / Restricción
truckId	string	Sí	ID del camión (referencia a truck_master de SAP)
capacityWeight	number	Sí	Capacidad en peso extraída en el paso anterior
capacityVolume	number	Sí	Capacidad en volumen extraída en el paso anterior

JSON

```
{
  "planStatusId": 1,
  "distributorId": 1,
  "planDate": "2026-07-15T00:00:00.000Z",
  "plannedOrderCount": 0,
  "plannedTruckCount": 2,
  "createdBy": "D. Rojas - 6055",
  "trucks": [
    {
      "truckId": 880012,
      "capacityWeight": 5000.00,
      "capacityVolume": 18.50
    },
    {...}
  ]
}
```

A dispatch-plan() 2.1

Controlador responsable de recibir el payload, validar mediante class-validator que planned_truck_count coincida con la longitud de trucks, y enviar los datos al Planning Service.

B createDispatchPlan() 2.2

Persiste la cabecera en la tabla dispatch_plan. El servicio es responsable de autocompletar los campos de estado y fechas de auditoría que no vienen del frontend:

- **Mapeo por defecto en BD:**
 - plan_status_id: Se busca y asigna internamente el ID correspondiente al estado "DRAFT".
 - plan_status: "DRAFT"
- **Parámetro de salida:** dispatch_plan_id (PK autogenerada).

C createPlanningTruck() 2.3

Bucle (loop) que inserta cada elemento del array trucks en la tabla planning_truck.

- **Parámetro de entrada:** dispatch_plan_id y TruckPlanDto.
- **Mapeo por defecto en BD para inicialización:**
 - assigned_weight: es la copia de la capacidad del peso
 - assigned_volume: es la copia de la capacidad del volumen
 - is_included_in_routing: true

Response Principal

Retorna la confirmación y los datos base del borrador guardado.

JSON

```
{
  "success": true,
  "code": 200,
  "message": "El plan de despacho se ha guardado en borrador correctamente.",
  "data": {
    "dispatchPlanId": 899,
    "planStatus": "DRAFT",
    "trucksSaved": 2
  }
}
```

03 Seleccionar Filtro

Endpoint.

Tipo (HTTP) POST /orders-by-filter

Especificación de DTOs y funciones

Request Principal (POST /orders-by-filter)

El Frontend debe enviar el estado completo de las selecciones del usuario. Esto incluye los filtros base de la planificación y los nuevos arreglos dinámicos seleccionados en los dropdowns de la interfaz.

Tabla de atributos FilterOrdersDto

Atributo	Tipo	Oblig.	Descripción / Restricción
distributorId	number	Sí	ID de la distribuidora origen.
channelId	number	No	ID del canal base (si aplica filtro individual).
deliveryDateFrom	Date	Sí	Rango de fecha de las órdenes (Desde).
deliveryDateTo	Date	Sí	Rango de fecha de las órdenes (Hasta).
productType	number	No	ID del tipo de producto (ej. frío/seco).
paymentType	number	No	ID del tipo de pago.
company	number	No	ID de la sociedad/empresa.
cities	number[]	No	Lista de IDs de ciudades seleccionadas.
channels	number[]	No	Lista de IDs de canales seleccionados (selección múltiple).
markets	number[]	No	Lista de IDs de mercados.
zones	number[]	No	Lista de IDs de zonas (Norte, Sur, etc.).
employees	number[]	No	Lista de IDs de vendedores.

Ejemplo JSON (Request)

JSON

```
{
  "distributorId": 1,
  "channelId": 1,
  "deliveryDateFrom": "2026-07-10T00:00:00.000Z",
  "deliveryDateTo": "2026-07-11T23:59:59.000Z",
  "productType": 1,
  "paymentType": 0,
  "company": 2,
  "cities": [5, 12],
  "channels": [1, 2],
  "markets": [3],
  "zones": [1, 4],
  "employees": []
}
```

A. ordersByFilter(filterOrdersDto) 2.2 y 2.3

El Gateway Controller envía la petición al Planning Controller, quien a su vez delega la lógica al Planning Service.

- **Parámetro de entrada:** filterOrdersDto.

B. orderByFilterMs(:id) 2.3 y listOrders 2.4

El Planning Service realiza una petición hacia MS_Sales.

- **Respuesta de MS_Sales:** Un arreglo de órdenes de venta (sales_order y sus sales_order_item).

JSON.

```
[
  {
    "orderId": 1,
    "ownerId": 1,
    "ownerName": "cliente 1",
    "totalWeight": 30,
    "totalVolume": 0.5,
    "deliveryWindowStart": "08:00",
    "deliveryWindowEnd": "10:00",
    "totalNeto": 15.00,
    "deliveryPointId": 123,
    "employeeId": 456,
    "employeeName": "John Doe"
    "isCutoffTime": true
  },
  {...}
]
```

Response Principal

JSON.

```
[{
  "totalCustomers": 12,
  "totalOrders": 120,
  "totalWeight": 3500,
  "channelId": 1,
  "cityId": 3,
  "marketId": null,
  "zoneId": null,
  "listOrder": [
    {
      "orderId": 1,
      "ownerId": 1,
      "ownerName": "Cliente 1",
      "totalWeight": 30,
      "totalVolume": 0.5,
      "deliveryWindowStart": "08:00",
      "deliveryWindowEnd": "10:00",
      "totalNeto": 15.00,
      "deliveryPointId": 123,
      "employeeId": 456,
      "employeeName": "John Doe"
    },
    {...}
  ],
  "listOrderCutOffTime": [
    {
      "orderId": 1,
      "ownerId": 2,
      "ownerName": "Cliente 2",
      "totalWeight": 30,
      "totalVolume": 0.5,
      "deliveryWindowStart": "08:00",
      "deliveryWindowEnd": "10:00",
      "totalNeto": 15.00,
      "deliveryPointId": 123,
      "employeeId": 456,
      "employeeName": "John Doe"
    },
    {...}
  ]
},
{...}]
```

04 Crear Órdenes de Planificación.

Enpoint

Tipo : (HTTP) POST /dispatch-plan-order

Especificación de DTOs y funciones

El Frontend envía el ID del plan de despacho previamente creado (en estado DRAFT) y la lista plana de todos los pedidos seleccionados (tanto regulares como fuera de corte) para ser consolidados.

Ejemplo del Body

```
{
  "dipatchPlanId": 8900,
  "listData": [
    {
      "totalCustomers": 12,
      "totalOrders": 120,
      "totalWeight": 3500,
      "channelId": 1,
      "cutoffTime": "08:00",
      "listOrder": [
        {
          "orderId": 1,
          "ownerId": 2,
          "ownerName": "Cliente 2",
          "totalWeight": 30,
          "totalVolume": 0.5,
          "deliveryWindowStart": "08:00",
          "deliveryWindowEnd": "10:00",
          "totalNeto": 15.00,
          "deliveryPointId": 123,
          "employeeId": 456,
          "employeeName": "John Doe"
        }
      ],
      "listOrderCutOffTime": [
        {
          "orderId": 1,
          "ownerId": 3,
          "ownerName": "Cliente 3",
          "totalWeight": 30,
          "totalVolume": 0.5,
          "deliveryWindow_start": "08:00",
          "deliveryWindow_end": "10:00",
          "totalNeto": 15.00,
          "deliveryPoinId": 123,
          "employeeId": 456,
          "employeeName": "John Doe"
        }
      ]
    }
  ]
}
```

A. createDispatchPlanOrder(Body) 4.1 y 4.2

El Gateway Controller y el Planning Controller reciben la petición, validan el DTO y transfieren la ejecución de la lógica de negocio al Planning Service.

B. groupByDeliveryPoint() 4.3

Función en memoria dentro del servicio que procesa el arreglo orders. Su objetivo es la **unificación de puntos de entregas**. Si dos pedidos distintos van al mismo delivery_point_id se agrupan en una única cabecera de parada para el camión.

C. createDispatchDeliveryPoints() 4.4

Se utiliza el modelo de TypeOrm para persistir los grupos generados en el paso anterior dentro de la tabla dispatch_delivery_point. Cada registro aquí representa una parada física.

- **Mapeo por defecto en BD:**
 - dispatch_plan_id: Viene del Request.
 - delivery_point_id: Agrupado en 4.3.
 - forced_planning_truck_id: Se inicializa en null (a menos que el planificador ya haya forzado una asignación manual).
- **Parámetro de salida:** Retorna un arreglo con los IDs autogenerados de dispatch_delivery_point, vinculados a su respectivo delivery_point_id.

Ejemplo del parámetro en JSON .

```
{
  "dispatchPlanId": 899,
  "deliveryPointId": 5501,
  "ownerId": 452,
  "ownerName": "Cliente propietario 1",
  "priority": null,
  "zone_id": null,
  "warehouse_destination_id": null,
  "deliveryWindowStart": "08:00",
  "deliveryWindowEnd": "12:00",
  "totalNeto": 5902.22,
  "totalWeight": 150.50,
  "totalVolume": 2.5
}
```

Ejemplo de la respuesta.

```
[{"id": 90,}]
```

E. createCandidateOrder() 4.4

Se utiliza el model de candidate_order para realizar un bulkCreate y guardar todas las órdenes de venta individuales vinculadas a sus respectivas cabeceras de punto de entrega.

- **Mapeo en BD:**
 - dispatch_delivery_point_id: Obtenido en el paso C.
 - status: "PENDING" (o el estado inicial que manejes para candidatos).

Ejemplo parametro JSON.

```
[
  {
    "dispatchDeliveryPointId": 899,
    "salesOrderId": 9201,
    "distributorId": 1,
    "deliveryWindowStart": "08:00",
    "deliveryWindowEnd": "12:00",
    "totalWeightKg": 150.50,
    "totalVolumeM3": 2.50,
    "typeOrder": "DELIVERY", // PUEDE SER ENTREGA, DEVOLUCION, TRASLADO
  },
  {...}
]
```

Response.

```
{"totalCreated":2}
```

Response Principal

El servicio finaliza la transacción de TypeOrm de forma exitosa y responde al Frontend.

Tabla DTO: DispatchPlanOrderResponseDto

Atributo	Tipo TypeScript	Descripción
success	boolean	Bandera de éxito de la operación.
deliveryPointsCreated	number	Cantidad de puntos de entrega unificados creados.
candidateOrdersCreated	number	Cantidad total de órdenes candidatas insertadas.
message	string	Mensaje de retroalimentación para el usuario.

Ejemplo JSON (Response)

JSON

```
{
  "success": true,
  "code": 200,
  "message": "Se han creado 2 pedidos en 1 punto de entrega exitosamente.",
  {
    "deliveryPointsCreated": 1,
    "candidateOrdersCreated": 2,
  }
}
```


05 Obtener Traslados

Endpoint.

Tipo : (HTTP) GET/transfers

3. Especificación de DTOs y funciones

Request Principal (GET/transfers)

El Frontend envía unos Query Params solicitando los traslados asociados a un distribuidor específico.

Tabla de atributos getTransfersDto

Atributo	Tipo	Oblig.	Descripción / Restricción
distributorId	string	Sí	ID de la distribuidora para la cual se consultan los traslados.
startDate	Date	no	Rango de fecha (Inicio)
endDate	Date	no	Rango de fecha (Fin)
toWarehouse	number	no	Almacén de destino
typeMovement	string	Si	Tipo de movimiento (entrega/recojo)

A. Ejemplo JSON (Request)

JSON

```
{
  "distributorId": 1,
  "startDate": "2026-07-16",
  "startEnd": "2026-07-15",
  "toWarehouse": 12,
  "typeMovement": "PICKUP" // DELIVERY
}
```

B. Funciones de la Secuencia

1. **getTransfers(getTransfersDto) 5.1:** El Gateway Controller intercepta el GET /transfers, valida el token de autenticación y transfiere el Query Params(GetTransfersRequestDto) al Planning Controller.
2. **getTransfers(getTransfersDto) 5.2:** El controlador valida que el DTO cumpla con las reglas de negocio (usando class-validator) y delega la operación al Planning Service.

3. **getTransfers(getTransfersDto)** 5.3: El Planning Service actúa como cliente. Formatea el DTO según el microservicio externo (MS Service) y realiza la petición asíncrona para obtener la data.

D. Response Principal (listTransfer)

El microservicio externo responde con la lista de traslados. El Planning Service retorna este arreglo al Frontend.

Tabla DTO: TransferResponseDto

Atributo	Tipo TypeScript	Descripción
transferId	string	ID único del traslado.
originWarehouseId	string	Almacén origen.
originWarehouseName	string	Nombre del almacén origen.
destinationWarehouseId	string	Almacén destino.
destinationWarehouseName	string	Nombre del almacén destino.
destinationWarehouseDeliveryWindowsStart	string	Inicio de la ventana horaria
destinationWarehouseDeliveryWindowsEnd	string	Fin de la ventana horaria
totalWeight	number	Peso del traslado en KG.
totalVolume	number	Volumen del traslado en M3.
status	string	Estado del traslado.
requestedDate	Date	Fecha de la solicitud.

Ejemplo JSON (Response)

JSON

```
{
  "success": true,
  "code": 200,
  "data": [
    {
      "transferId": 90012,
      "originWarehouseId": 11,
      "originWarehouseName": "Sucursal Norte",
      "destinationWarehouseId": 5,
      "destinationWarehouseName": "Sucursal Sur",
      "destinationWarehouseDeliverywindowStart": "08:00",
      "destinationWarehouseDeliverywindowEnd": "12:00",
      "totalWeight": 1250.50,
      "totalVolume": 14.8,
      "status": "CONFIRMED",
      "requestedDate": "2026-07-16T09:00:00.000Z"
    },
    { ... }
  ]
}
```

06 Obtener Devoluciones

Enpoint.

Tipo: (HTTP) GET /returns

Especificación de DTOs y funciones

Request Principal (GET /returns)

Se recibe los parámetros de consulta (*Query Parameters*) en la URL.

Tabla de atributos GetReturnsQueryDto

Atributo	Tipo TypeScript	Validación (class-validator)	Oblig.	Descripción / Restricción
distributorId	string	@IsString()	Sí	ID de la distribuidora para consultar sus devoluciones en espera de planificación.
typeReturn	string	@IsString()	Sí	Tipo de devolución ya sea de entrega o recojo (delivery/pickup)
startDate	Date	@IsDate()	No	Rango de fecha (Inicio)
endDate	Date	@IsDate()	No	Rango de fecha (Fin)

Ejemplo Query Params.

```
{
  "distributorId": 1,
  "typeReturn": "delivery",
  "startDate": "2026-07-16",
  "endDate": "2026-07-16",
}
```

A. Funciones de la Secuencia

1. **getReturns(queryParams) 6.1:** El Gateway Controller recibe el GET /returns y los query params
2. **getReturns(queryParams) 6.2:** El Planning Controller valida la estructura los parametros de la query.
3. **getReturns(queryParams) 6.3:** El Planning Service solicita la lista de devoluciones con los parametros de la query.

C. Response Principal (listReturns)

El Planning Service recibe los datos del microservicio externo y los envía a través de los controladores de vuelta al Frontend.

Tabla DTO: ReturnResponseDto

Atributo (camelCase)	Tipo TypeScript	Descripción
returnId	string	ID único de la devolución.
ownerId	string	ID del cliente.
owner Name	string	Nombre del cliente.
deliveryWindowStart	string	Inicio de la ventana horaria
deliveryWindowEnd	string	Fin de la ventana horaria
totalWeight	number	Peso de la devolución en KG.
totalVolume	number	Volumen de la devolución en M3.
status	string	Estado de la devolución.
refundDate	date	Fecha de la devolucion

Ejemplo JSON (Response)

JSON

```
{
  "success": true,
  "code": 200,
  "data": [
    {
      "refundId": 231,
      "ownerId": 452,
      "ownerName": "Tienda Norte",
      "deliveryWindowStart": "08:00",
      "deliveryWindowEnd": "12:00",
      "totalWeight": 1250.50,
      "totalVolume": 14.8,
      "status": "PENDING",
      "refundDate": "2026-07-16T08:24:39.000Z",
    },
    { ... }
  ]
}
```

07 Agregar Devoluciones y Transferencias a la Planificación

Endpoint.

Tipo: (HTTP) POST /add-candidate-orders

Especificación de DTOs y funciones

Request Principal (POST /add-candidate-orders)

El Frontend envía las listas de transferencias y devoluciones que el usuario decidió incluir.

Tabla de atributos AddTransferRefundDto.

Atributo	Tipo	Oblig.	Descripción / Restricción
dispatchPlanOrderId	number	Sí	ID del Dispatch Plan Order.
transfers	TransferCandidateDto[]	No	Lista de traslados a agregar.
returns	ReturnCandidateDto[]	No	Lista de devoluciones a agregar.

Sub-DTO: TransferCandidateDto (Traslados)

Este DTO representa la carga de un traslado que se agregará como un candidato a la ruta.

Atributo	Tipo	Oblig.	Descripción / Regla de Relación
transferId	number	Sí	ID único del traslado del microservicio de inventarios.
destinationDeliveryPointId	number	Sí	ID del punto de entrega (coordenadas GPS) del almacén de destino.
distributorId	number	Sí	ID de la distribuidora de origen que despacha el traslado.
deliveryWindowStart	string	No	Hora de inicio permitida para la descarga (Formato HH:MM:SS).
deliveryWindowEnd	string	No	Hora límite permitida para la descarga (Formato HH:MM:SS).
totalWeight	number	Sí	Peso total de la mercancía a trasladar (Decimal).
totalVolume	number	Sí	Volumen total de la mercancía en metros cúbicos (Decimal).
priority	number	Sí	Nivel de prioridad de entrega en la secuencia de ruta.
notes	string	No	Observaciones del planificador para el chofer.
createdBy	string	Sí	ID/Nombre del planificador que realiza la acción.

Sub-DTO: ReturnCandidateDto (Devoluciones/Recojos)

Atributo	Tipo	Oblig.	Descripción / Regla de Relación
refundOrderId	number	Sí	ID único de la orden de devolución del sistema de ventas.
deliveryPointId	number	Sí	ID del punto de entrega del cliente donde se recogerá el producto.
distributorId	number	Sí	ID de la distribuidora asignada para recibir la devolución.
deliveryWindowStart	string	No	Hora de inicio de la ventana de atención del cliente (Formato HH:MM:SS).
deliveryWindowEnd	string	No	Hora de fin de la ventana de atención del cliente (Formato HH:MM:SS).
totalWeight	number	Sí	Peso total estimado de la mercancía de retorno.
totalVolume	number	Sí	Volumen total estimado de la mercancía de retorno.
priority	number	Sí	Prioridad asignada para el recojo.
notes	string	No	Instrucciones específicas de recojo (ej: "Llevar caja sellada").
createdBy	string	Sí	ID/Nombre del planificador que realiza la acción.

JSON

```
{
  "dispatchPlanId": 778899,
  "transfers": [
    {
      "transferId": 90012,
      "originWarehouseId": 11,
      "originWarehouseName": "Sucursal Norte",
      "destinationWarehouseId": 5,
      "destinationWarehouseName": "Sucursal Sur",
      "destinationWarehouseDeliverywindowStart": "08:00",
      "destinationWarehouseDeliverywindowEnd": "12:00",
      "totalWeight": 1250.50,
      "totalVolume": 14.8,
      "requestedDate": "2026-07-16T09:00:00.000Z"
    }
  ],
  "returns": [
    {
      "refundId": 231,
      "ownerId": 452,
      "ownerName": "Tienda Norte",
      "deliveryWindowStart": "08:00",
      "deliveryWindowEnd": "12:00",
      "totalWeight": 1250.50,
      "totalVolume": 14.8,
      "refundDate": "2026-07-16T08:24:39.000Z",
    }
  ]
}
```

GroupByDeliveryPoint 7.2a

Funcion que agrupa los traslados y devoluciones por punto de entrega y retonar una nueva lista.

Parametro de entrada: **AddTransferRefundDto**

Parametro de salida: **groupTransferRedundDto.**

Json

```
[
  {
    "deliveryPointId": 5501,
    "returns": [
      {
        "refundId": 231,
        "ownerId": 452,
        "ownerName": "Tienda Norte",
        "deliveryWindowStart": "08:00",
        "deliveryWindowEnd": "12:00",
        "totalWeight": 1250.50,
        "totalVolume": 14.8,
        "status": "PENDING",
        "refundDate": "2026-07-16T08:24:39.000Z",
      }
    ]
  },
  {
    "deliveryPointId": 5501,
    "transfers": [
      {
        "transferId": 90012,
        "originWarehouseId": 11,
        "originWarehouseName": "Sucursal Norte",
        "destinationWarehouseId": 5,
        "destinationWarehouseName": "Sucursal Sur",
        "destinationWarehouseDeliverywindowStart": "08:00",
        "destinationWarehouseDeliverywindowEnd": "12:00",
        "totalWeight": 1250.50,
        "totalVolume": 14.8,
        "status": "CONFIRMED",
        "requestedDate": "2026-07-16T09:00:00.000Z"
      }
    ]
  }
]
```

B. createCandidateOrder() 7.3

Dentro del bucle iterativo de transfers o refund, se generará un bulkCreate (o inserciones individuales) en la tabla candidate_order.

- **Mapeo DB:** El campo transfer_id recibe el valor, mientras que sales_order_id y refund_order_id quedan en null. El delivery_point_id se asume de la entidad destino.

JSON

```
{
  "dispatchDeliveryPointId": 778899,
  "salesOrderId": 9201,
  "deliveryWindowStart": "08:00",
  "deliveryWindowEnd": "12:00",
  "distributorId": 1,
  "totalWeight": 1250.50,
  "totalVolume": 14.80,
  "refundOrderId": null,
  "typeOrder": "DELIVERY",
  "transferId": 90012
}
```

D. addCandidateOrderResponse 7.5

El servicio confirma que las inserciones fueron exitosas y retorna el resumen al Frontend.

Tabla DTO: AddCandidateOrderResponseDto

Atributo	Tipo TypeScript	Descripción
success	boolean	Indica si la operación transaccional se completó.
transfersAdded	number	Cantidad de transferencias vinculadas a la planificación.
refundsAdded	number	Cantidad de devoluciones vinculadas a la planificación.
message	string	Mensaje informativo de confirmación.

Ejemplo JSON (Response)

JSON

```
{
  "success": true,
  "code": 200,
  "message": "Se han agregado correctamente las devoluciones a la planificación.",
  "data": {
    "transfersAdded": 0,
    "refundsAdded": 1,
  }
}
```

08 Obtener Puntos de entregas.

Endpoint.

Tipo: (HTTP) GET/dispatch-plans/:dispatchPlanId/delivery-points

Especificación de DTOs y funciones

Request Principal (GET/dispatch-plans/:dispatchPlanId/delivery-points)

El Frontend envía unos Query Params solicitando los puntos de entrega asociados a un plan de despacho específico.

Tabla de atributos GetDeliveryPointsRequestDto

Atributo	Tipo TypeScript	Validación (class-validator)	Oblig.	Descripción / Restricción
dispatchPlanId	string	@IsString()	Sí	ID del plan de despacho para el cual se consultan los traslados.

A. Ejemplo JSON (Request)

JSON

```
{
  "dispatchPlanId": 1,
}
```

Detalle del Flujo de Ejecución

A. **getDeliveryPoints(dispatchPlanId)** (8.1 y 8.2)

El Gateway Controller y el Planning Controller reciben la petición HTTP GET, validan los parámetros de entrada y delegan la ejecución al Planning Service.

B. **groupByDeliveryPoint(dispatchPlanId)** (8.3)

El **Planning Service** consulta a la entidad **Dispatch Delivery Point DB** para obtener los registros de los puntos de entrega.

C. **Obtención de Datos en Bucle (Loop 8.4 - 8.5).**

Para cada registro de orden recuperado, la capa de persistencia resuelve recursivamente las relaciones hacia las tablas externas:

- **groupByDeliveryPoint(deliveryPointId)** (8.4): Obtiene los detalles geográficos y de ubicación de la entidad **delivery_point DB**.
- **groupByCustomer(ownerId)** (8.5): Obtiene la información de cliente asociado desde la entidad **customers DB**.

D. Response Principal

El servicio procesa los datos y retorna una lista detallada de los puntos de entrega y sus relaciones.

Tabla DTO: GetDeliveryPointsResponseDto

Atributo	Tipo	Descripción
dispatchDeliveryPointId	number	ID primario del registro (dispatch_delivery_point.id).
dispatchPlanId	number	ID de la planificación asociada.
deliveryPointId	DeliveryPointDto	Objeto con información detallada del punto de entrega.
ownerId	number	ID del cliente propietario
ownerName	string	Nombre del cliente propietario
priority	number null	Prioridad asignada a la parada.
deliveryWindowStart	string null	Hora de inicio de la ventana de atención
deliveryWindowEnd	string null	Hora de fin de la ventana de atención
totalNeto	number	Monto total neto consolidado del punto de entrega.
totalWeight	number	Peso total consolidado
totalVolume	number	Volumen total consolidado
forcedPlanningTruckId	number null	ID del camión si fue asignado manualmente.

Ejemplo JSON (Response)

JSON

```
{
  "success":true,
  "code": 200,
  "data": [
    {
      "dispatchDeliveryPointId": 1929,
      "dispatchPlanId": 1223,
      "ownerId": 1,
      "ownerName": "cliente padre 1",
      "customerId": 15,
      "customerName": "cliente hijo 1",
      "priority": 1,
      "deliveryWindowStart": "08:00",
      "deliveryWindowEnd": "12:00",
      "totalWeight": 150.50,
      "totalVolume": 2.50,
      "totalNeto": 5902.22,
      "forcedPlanningTruckId": null,
      "deliveryPoint": {
        "deliveryPointId": 123,
        "addressText": "Av. Las Américas #1234",
        "latitude": -21.5338,
        "longitude": -64.7339
      }
    },
    {...}
  ]
}
```

09 Optimización Multiple de Rutas (Google Route Optimization)

Enpoint.

Tipo: (HTTP) POST/optimization-route

Especificación de DTOs y Funciones

Request Principal (GET /optimization-route)

El Frontend envía únicamente el ID del plan de despacho que desea optimizar.

Tabla de atributos OptimizationRouteDto

Atributo	Tipo	Oblig.	Descripción / Restricción
dispatchPlanId	number	Sí	ID (PK) del planificacion a procesar.
forcedAssignments	array	No	Pins manuales deliveryPointId → planningTruckId. Vacío en la 1ra optimización; se llena a medida que el planificador mueve paradas.

Ejemplo JSON (Request)

JSON

```
{
  "dispatchPlanId": 778899,
  "forcedAssignments": [
    { "deliveryPointId": 401, "planningTruckId": 2 },
    { "deliveryPointId": 405, "planningTruckId": 1 }
  ]
}
```

Funciones del Flujo.

9.1 optimizationRoute(dto) — El Gateway recibe el request y lo reenvía al Controller.

9.2 optimizationRoute(dto) — El Controller valida y lo pasa al Planning Service.

9.3 – 9.4 getDispatchPlan(dispatchPlanId) — Lee de la BD las paradas del plan, sus candidate orders y los camiones asignados. No cambia respecto a la versión anterior.

9.5 formatterToGoogleRoute(dispatchPlan, forcedAssignments) — Transforma la data interna al payload de Google. Por cada punto con pin, agrega allowedVehicleIndices al shipment (el **índice** del camión dentro de vehicles[], no su id).

9.6 – 9.7 getRoutes(googleDto) — Llama a Google Route Optimization. Devuelve rutas, polilíneas y métricas.

9.8 buildRouteUIResponse(googleRoutes, dispatchPlan) — **En memoria** (no escribe BD): cruza la respuesta de Google con el plan, suma peso y Bs por ruta, y arma la respuesta. Cada ruta se identifica por planningTruckId (todavía no existe routeId).

9.9 return listRoutes — Devuelve el resultado al frontend.

A. getDispatchPlan(dispatchPlanId) 9.3 y return dispatchPlan 9.4

El Planning Service realiza una consulta a la base de datos (con varios JOINS) para recuperar las paradas (dispatch_delivery_point), sus pedidos candidatos (candidate_order), y los camiones asignados (planning_truck)

Tabla principal: InternalDispatchPlanDto

Atributo	Tipo	Descripción de la Relación / Extracción
dispatchPlanId	number	ID (PK) del plan de despacho consultado.
dispatchDeliveryPoints	InternalDispatchDeliveryPointsDto[]	Relación 1 a Muchos. Son las paradas asociadas al plan.
planningTruck	InternalTruckDto[]	Relación 1 a Muchos. Son los camiones asignados para este plan.

Sub-DTO: InternalDispatchDeliveryPointDto (Paradas del plan)

Atributo	Tipo	Descripción de la Relación / Extracción
priority	number	Prioridad asignada a la parada.
deliveryPoint	InternalDeliveryPointDto	Objeto anidado obtenido al cruzar dispatch_plan_order.delivery_point_id con la tabla delivery_point.
deliveryWindow	InternalWindowDto	Objeto construido a partir de los campos de hora de la cabecera de la parada.
candidateOrders	InternalCandidateDto[]	Relación 1 a Muchos. Todos los pedidos individuales agrupados en esta parada.

Sub-DTO: InternalDeliveryPointDto (Coordenadas de la Parada)

Atributo	Tipo	Descripción de la Relación / Extracción
deliveryPointId	number	ID de la ubicación física (Cliente o Almacén destino).
latitude	number	Latitud GPS extraída de la tabla caché de SAP.
longitude	number	Longitud GPS extraída de la tabla caché de SAP.

Sub-DTO: InternalWindowDto (Ventana de tiempo)

Atributo	Tipo	Descripción de la Relación / Extracción
start	string	Hora de inicio permitida (Formato HH:MM).
end	string	Hora de cierre permitida (Formato HH:MM).

Sub-DTO: InternalCandidateDto (Detalle de los pedidos)

Atributo	Tipo	Descripción de la Relación / Extracción
saleOrderId	number	ID interno del pedido candidato.
totalWeight	number	Peso total del pedido específico.
totalVolume	number	Volumen total del pedido específico.

Sub-DTO: InternalTruckDto (Camiones disponibles para la ruta)

Atributo	Tipo	Descripción de la Relación / Extracción
truckId	number	FK que apunta a la tabla maestra truck. Se usará como label en Google.
assignedWeight	number	Peso asignado previamente (o cero si apenas se va a calcular).
assignedVolume	number	Volumen asignado previamente (o cero).

JSON Interno

```
{
  "dispatchPlanId": 778899,
  "dispatchDeliveryPoints": [
    {
      "priority": 1,
      "deliveryPoint": {
        "deliveryPointId": 123,
        "latitude": -21.5338,
        "longitude": -64.7339
      },
      "deliveryWindow": {"start": "08:00", "end": "12:00"},
      "candidateOrders": [
        {
          "saleOrderId": 1,
          "totalWeight": 30,
          "totalVolume": 0.5
        },
        {...}
      ]
    }
  ],
  "planningTruck": [
    {
      "truckId": 1,
      "assignedWeight": 5,
      "assignedVolume": 0.1
    }
  ]
}
```

B. formatterToGoogleRoute(dispatchPlan) 9.5

Esta función formatea los datos extraídos de tu base de datos a la estructura que exige Google Route Optimization.

- *Mapeo Clave:* Los truckId se mapean a los vehicles[].label. Las paradas consolidadas se suman (peso/volumen) y se envían como shipments.

Especificación del Payload

Propiedad (Ruta JSON)	Tipo	Oblig.	Descripción / Propósito en el Motor de Google
populatePolylines	boolean	Sí	Bandera que le indica a Google si debe generar y retornar las polilíneas codificadas de las rutas calculadas para pintarlas en el mapa.
model	object	Sí	Objeto principal contenedor que define todo el problema logístico (flota y pedidos).
model.vehicles[]	array[object]	Sí	Arreglo que define la flota de camiones disponibles para realizar las rutas.
model.vehicles[].label	string	Sí	Identificador único o etiqueta del vehículo (ej. la placa o código)

			interno del camión).
model.vehicles[].loadLimits	object	Sí	Objeto que define las capacidades máximas de carga física del camión.
model.vehicles[].loadLimits.peso_kg.maxLoad	string	Sí	Límite máximo de peso en kilogramos que soporta el vehículo (en formato string).
model.vehicles[].loadLimits.volumen_m3.maxLoad	string	Sí	Límite máximo de volumen en metros cúbicos que soporta el vehículo.
model.vehicles[].costPerKilometer	number	No	Costo económico por kilómetro recorrido, utilizado por Google para optimizar el costo total de la ruta.
model.vehicles[].costPerHour	number	No	Costo económico por hora operando, usado para equilibrar tiempos y costos.
model.vehicles[].startLocation	object	Sí	Coordenadas geográficas (latitude, longitude) del punto de partida del

			camión (Almacén / Centro de distribución).
model.vehicles[].endLocation	object	Sí	Coordenadas geográficas (latitude, longitude) del punto de llegada del camión al finalizar su ruta.
model.shipments[]	array[object]	Sí	Arreglo que representa la lista de pedidos o puntos de entrega que el motor debe planificar.
model.shipments[].label	string	Sí	Etiqueta identificadora única del envío (ej. número de orden o venta).
model.shipments[].loadDemands	object	Sí	Demandas de capacidad que el pedido restará al vehículo que lo transporte.
model.shipments[].loadDemands.peso_kg.amount	string	Sí	Peso total del pedido en kilogramos (formato string).
model.shipments[].loadDemands.volumen_m3.amo unt	string	Sí	Volumen total del pedido en metros cúbicos.

<code>model.shipments[].deliveries[]</code>	<code>array[object]</code>	Sí	Arreglo con las acciones de entrega a realizar para este envío.
<code>model.shipments[].deliveries[].arrivalLocation</code>	<code>object</code>	Sí	Coordenadas GPS exactas (latitude, longitude) del cliente o punto de entrega.
<code>model.shipments[].deliveries[].duration</code>	<code>string</code>	Sí	Tiempo estimado de parada o descarga en el punto (expresado en segundos, ej. "6000s").

JSON (Request hacia Google API)

```
{
  "populatePolylines": true,
  "model": {
    "vehicles": [
      {
        "label": "camion-volvo-xyz-88",
        "loadLimits": {
          "peso_kg": { "maxLoad": "10000" },
          "volumen_m3": { "maxLoad": "30" },
          "costPerKilometer": 10,
          "costPerHour": 0.75,
          "startLocation": { "latitude": -17.7833, "longitude": -63.1821 },
          "endLocation": {
            "latitude": -17.7833,
            "longitude": -63.1821
          }
        }
      }
    ],
    "shipments": [
      {
        "label": "PE1",
        "loadDemands": {
          "peso_kg": { "amount": "4500" },
          "volumen_m3": { "amount": "10" }
        },
        "deliveries": [
          {
            "arrivalLocation": { "latitude": -17.7689, "longitude": -63.1705 },
            "duration": "6000s"
          }
        ]
      }
    ]
  }
}
```

C. getRoutes(googleRouteOptimizationDto) 9.6

Se realiza la petición HTTP a Google. Google retorna la respuesta optimizada con múltiples rutas (una por cada camión utilizado), polilíneas, y métricas.

return googleRoutes 9.7

Especificacion del Payload

Ruta en el JSON	Tipo	Descripción / Propósito para el Backend
routes[]	Array	Contenedor principal de las rutas optimizadas que devolvió Google.
routes[].vehicleLabel	String	Identificador del camión. El backend lo usa para relacionar la ruta con el camión o viaje correcto.
routes[].routePolyline.points	String	Cadena codificada que almacena la geometría exacta del mapa para renderizarla en el Frontend.
routes[].metrics.travelDistanceMeters	Number	Distancia total en metros que recorrerá el camión en esta ruta.
routes[].metrics.travelDuration	String	Tiempo total estimado de manejo (se parsea de string a segundos para la BD).
routes[].routeTotalCost	Number	Costo operativo monetario total estimado calculado por Google para la ruta.
routes[].visits[]	Array	Secuencia ordenada de paradas que el backend recorrerá mediante un loop (forEach / for...of).
routes[].visits[].shipmentLabel	String	Etiqueta del pedido que permite identificar a qué parada específica de tu BD corresponde esta visita.
routes[].visits[].startTime	String	Hora estimada de llegada (ETA) calculada por Google para esta parada específica.

Respuesta de Google en Json.

```
{
  "routes": [ // Lista de rutas optimizadas
    {
      "vehicleLabel": "camion-1", // Etiqueta del camion.
      "routePolyline": { "points": "`lpkBhjs`KnEMnB..." }, //Polilineas codificada
      "metrics": {
        "travelDistanceMeters": 6349, // Distancia en metros que recorrera
        "travelDuration": "1233s" // Duración total estimada del viaje en segundos
      },
      "routeTotalCost": 64.04, // Costo operativo total estimado
      "visits": [ // Secuencia ordenada de paradas
        {
          "shipmentLabel": "PE1", // Etiqueta de cada punto de entrega
          "startTime": "1970-01-01T00:23:52Z" // Hora estimada de llegada
        }
      ]
    }
  ]
}
```

D. buildRouteUIResponse 9.8

Construida en memoria. Sin routeId porque no se persistió nada; cada ruta se identifica por planningTruckId.

Se itera cada ruta que devolvió Google y se cruza con el dispatchPlan (ya cargado en 9.4) para resolver nombre de cliente, peso y Bs.

Campo	Origen
planningTruckId	route.vehicleLabel de Google → planningTruckId (el label ES el truckId del camión planificado).
name	Generado por índice de iteración: "Ruta 1", "Ruta 2"...
encodePolyline	route.routePolyline.points.
etaTotalDistanceM	route.metrics.travelDistanceMeters.
etaTotalTimeS	route.metrics.travelDuration, limpiando la "s" ("4388s" → 4388).
totalCost	route.routeTotalCost.
totalRouteWeight	Σ total_weight de los dispatchDeliveryPoint de las visitas de esta ruta.
totalRouteBs	Σ total_neto de los dispatchDeliveryPoint de las visitas de esta ruta.
visits	route.visits[] de Google, mapeadas (ver abajo).

VisitUIResponseDto (por parada)

Campo	Origen
sequence	Índice del array route.visits (1, 2, 3...).
dispatchDeliveryPointId	visit.shipmentLabel (el label ES el id del punto).
clientName	dispatch_delivery_point.owner_name (resuelto vía owner_id).
totalBs	dispatch_delivery_point.total_neto.
totalWeight	dispatch_delivery_point.total_weight.

JSON resultante

```
{
  "success": true,
  "dispatchPlanId": 778899,
  "routes": [
    {
      "planningTruckId": 1,
      "name": "Ruta 1",
      "encodePolyline": "`lpkBhjs`KnEMnBKxAGEwEC...",
      "etaTotalDistanceM": 6349,
      "etaTotalTimeS": 4388,
      "totalCost": 64.04,
      "totalRouteBs": 4200.00,
      "totalRouteWeight": 900.00,
      "visits": [
        {
          "sequence": 1,
          "dispatchDeliveryPointId": 405,
          "clientName": "Cliente Sur",
          "totalBs": 4200.00,
          "totalWeight": 900.00
        }
      ]
    },
    {
      "planningTruckId": 2,
      "name": "Ruta 2",
      "encodePolyline": "ah_lBnts`KpAa@dCy@...",
      "etaTotalDistanceM": 3120,
      "etaTotalTimeS": 2010,
      "totalCost": 28.90,
      "totalRouteBs": 11880.40,
      "totalRouteWeight": 2500.00,
      "visits": [
        {
          "sequence": 1,
          "dispatchDeliveryPointId": 401,
          "clientName": "Hipermaxi Norte",
          "totalBs": 11880.40,
          "totalWeight": 2500.00
        }
      ]
    }
  ]
}
```

E. Reoptimizaciones (2da vez en adelante)

El frontend guarda la respuesta y los pins en sessionStorage. Cuando el planificador mueve una parada y pide reoptimizar, manda el mismo endpoint con los pins acumulados:

```
{
  "dispatchPlanId": 778899,
  "forcedAssignments": [
    { "deliveryPointId": 401, "planningTruckId": 2 },
    { "deliveryPointId": 405, "planningTruckId": 1 }
  ]
}
```

Se ejecuta el mismo flujo 9.1 – 9.9 y devuelve el mismo formato de routes, recalculado. El frontend sobrescribe el cache.

Estado en sessionStorage

```
// key: "optimization:plan:778899"
{
  "dispatchPlanId": 778899,
  "updatedAt": "2026-07-23T14:32:10Z",
  "forcedAssignments": [
    {
      "deliveryPointId": 401,
      "planningTruckId": 2
    },
    {...}
  ],
  "routes": [
    {
      "planningTruckId": 1,
      "name": "Ruta 1",
      "visits": [ /* ... */ ]
    },
    {...}
  ]
}
```

Este ciclo se repite hasta que el planificador queda a gusto. Recién ahí, "Guardar y generar órdenes" persiste routes, route_delivery_points y crea las transport_order (con assigned_weight_kg / assigned_volume_m3).

10 Crear Rutas

Endpoint.

Tipo: (HTTP) POST /transport-orders/{dispatchPlanId}

10.1 Request (Frontend -> Gateway/Controller)

Especificacion de DTOs

Atributo	Tipo	Obligatorio	Descripción
dispatchPlanId	Number	Si	Id de la planificacion
routes	CreateRouteDto[]	Sí	Arreglo con la geometría, métricas y secuencia de paradas de las rutas calculadas.

Sub-DTO: CrateRouteDTO

Atributo	Tipo	Oblig.	Descripción
planningTruckId	number	Sí	ID (PK) del camión planificado
encodePolyline	string	Sí	Geometría de la ruta codificada retornada por Google.
etaTotalDistanceM	number	Sí	Distancia total recorrida en metros.
etaTotalTimeS	number	Sí	Tiempo total de viaje en segundos.
totalCost	number	Sí	Costo total operativo calculado.
visits	CreateRouteVisitDto[]	Sí	Secuencia ordenada de paradas asociadas a la ruta.

Sub-DTO: CreateRouteVisitDto

Atributo	Tipo	Obligatorio	Descripción
sequence	number	Sí	Orden secuencial de la visita (1, 2, 3...).
dispatchDeliveryPointId	number	Sí	ID (PK) de la parada

Ejemplo JSON (Request)

```
{
  "routes": [
    {
      "planningTruckId": 1,
      "encodePolyline": "`lpkBhjs`KnEMnB...",
      "etaTotalDistanceM": 6349,
      "etaTotalTimeS": 4388,
      "totalCost": 64.04,
      "visits": [
        {
          "sequence": 1,
          "dispatchDeliveryPointId": 405
        },
        {
          "sequence": 2,
          "dispatchDeliveryPointId": 401
        }
      ]
    }
  ]
}
```

Flujo del Proceso Backend

10.1 createTransportOrders(dto) — Gateway → Controller

Reenvía el request al controller. **No consulta BD.**

10.2 createTransportOrders(dto) — Controller → Service

Valida y delega al Planning Service. No consulta BD.

10.3.1 dispatch_delivery_points – findByDispatchPlanId()

Trae las paradas del plan con sus totales consolidados; son la base para resolver el peso y las ventas de cada ruta.

Formato JSON:

```
"dispatchDeliveryPoints":  
  [  
    {  
      "id": 405,  
      "deliveryPointId": 123,  
      "ownerId": 1,  
      "ownerName": "Cliente Sur",  
      "priority": 1,  
      "deliveryWindowStart": "08:00",  
      "deliveryWindowEnd": "12:00",  
      "totalWeightKg": 900,  
      "totalVolumeM3": 3.2,  
      "totalNeto": 4200.00,  
      "forcedPlanningTruckId": null  
    },  
    {  
      "id": 401,  
      "deliveryPointId": 456,  
      "ownerId": 2,  
      "ownerName": "Hipermaxi Norte",  
      "priority": 1,  
      "deliveryWindowStart": "07:00",  
      "deliveryWindowEnd": "11:00",  
      "totalWeightKg": 2500,  
      "totalVolumeM3": 6.4,  
      "totalNeto": 11880.40,  
      "forcedPlanningTruckId": null  
    }  
  ]
```

10.3.2 candidate_order – findByDeliveryPointId()

Trae los pedidos agrupados en cada parada; de acá salen los sale_order_id que irán a la orden de transporte.

Formato JSON:

```
"candidateOrder": [  
  {
```

```

{
  "id": 9001,
  "dispatchDeliveryPointId": 405,
  "salesOrderId": 556,
  "totalWeightKg": 900,
  "totalVolumeM3": 3.2,
  "typeOrder": "DELIVERY"
},
{
  "id": 9002,
  "dispatchDeliveryPointId": 401,
  "salesOrderId": 557,
  "totalWeightKg": 1500,
  "totalVolumeM3": 4.0,
  "typeOrder": "DELIVERY"
},
{
  "id": 9003,
  "dispatchDeliveryPointId": 401,
  "salesOrderId": 558,
  "totalWeightKg": 1000,
  "totalVolumeM3": 2.4,
  "typeOrder": "DELIVERY"
}
]

```

10.3.3 sales_order_item – findBySalesOrderId

Trae los ítems (producto + cantidad) de cada venta; son los que después se guardan como transport_order_sale_items.

Formato JSON:

```

"sales_order_item": [
  {

```

```

        "salesOrderId": 556,
        "productId": 123,
        "confirmedQty": 3
    },
    {
        "salesOrderId": 557,
        "productId": 200,
        "confirmedQty": 10
    },
    {
        "salesOrderId": 558,
        "productId": 201,
        "confirmedQty": 5
    }
]

```

10.3.4 getPlanningTrucks(dispatchPlanId) – findBySalesOrderId

Recupera los camiones planificados para el cálculo de los acumulados de peso y volumen.

Formato JSON:

```

"planning_truck": [
  {
    "id": 1,
    "truckId": 56,
    "assignedWeightKg": 3400,
    "assignedVolumeM3": 9.6,
    "isIncludedInRouting": true
  }
]

```

Resultado armado antes del LOOP de creacion:

Formato JSON:

```

{
  "deliveryPoints": [
    {
      "deliveryPointId": 405,
      "totalWeightKg": 900,
      "totalVolumeM3": 3.2,
      "sales": [

```

```

    {
      "saleOrderId": 556,
      "items": [
        {
          "productId": 123,
          "quantityMin": 3
        }
      ]
    }
  ],
  {
    "deliveryPointId": 401,
    "totalWeightKg": 2500,
    "totalVolumeM3": 6.4,
    "sales": [
      {
        "saleOrderId": 557,
        "items": [
          {
            "productId": 200,
            "quantityMin": 10
          }
        ]
      },
      {...}
    ]
  }
],
"planningTrucks": [
  {
    "planningTruckId": 1,
    "assignedWeightKg": 3400,
    "assignedVolumeM3": 9.6
  }
]
}

```

10.4 Loop Transaccional (Escritura BD)

Por cada elemento dentro del arreglo routes [] del request, se ejecutan las siguientes operaciones en **una única transacción**:

10.4.1 createTrip(truckId, driver=null, distributorId=null, status=PENDING)

Creamos el TRIP asociando el truckId, driver(Chofer) en null para posteriormente asignarle un chofer, y el status pendiente.

Formato JSON:

```

{
  "truckId": 56,
  "distributorId": 2,
  "driverEmployeeId": null,
  "helperEmployeeId": null,

```

```

    "status": "PENDING",
    "departureDate": "2026-07-27T08:00:00Z",
    "createdBy": "d.rojas"
  }

```

Formato JSON:

```

{
  "tripId": 50
}

```

10.4.2 createRoute(route)

Columna	Mapeo de Origen
dispatch_plan_id	dispatchPlanId
planning_truck_id	route.planningTruckId
engine	"Google Route Optimization"
encode_polyline	route.encodePolyline
eta_total_distance_m	route.etaTotalDistanceM
eta_total_time_s	route.etaTotalTimeS
total_cost	route.totalCost
is_selected	true
status	"CREATED"

Formato JSON:

```

{ "routeId": 3344 }

```

10.4.3 createRouteDeliveryPoints(routeId, visits)

Se crea el routeDeliveryPoint, se itera sobre **route.visits[]**

Formato JSON:

```

[
  {
    "routeId": 3344,
    "dispatchDeliveryPointId": 405,
    "sequence": 1,
    "isActive": true
  },
  {
    "routeId": 3344,
    "dispatchDeliveryPointId": 401,
    "sequence": 2,

```

```

        "isActive": true
    }
]

```

10.5 Respuesta del Endpoint (Response)

Especificación de DTO Response

Atributo	Tipo	Descripción
success	boolean	Estado de la operación.
code	number	Código de respuesta HTTP.
message	string	Descripción del resultado.
data.routes	number	Cantidad total de rutas creadas y persistidas.

Response Formato JSON:

```

{
  "success": true,
  "code": 200,
  "message": "Se crearon 2 rutas.",
  "data": {
    "routes": 2
  }
}

```


11 Obtener Ordenes de transporte

Listar ordenes de transporte:

Trae todas las ordenes de transporte generadas después de realizar la optimización.

Ruta: **GET** /dispatch-plans/{dispatchPlanId}/transport-orders

Parámetros de entrada

Parámetro	Tipo	Oblig.	Descripción
dispatchPlanId	number	No	ID del plan (viene en la URL). FK a dispatch_plan

Funciones:

A. getTransportOrders() 11.1

Controlador que recibe el dispatchPlanId, lo valida y lo pasa al Planning Service.

B. findByDispatchPlanId() 11.2

Trae del repo las órdenes del plan desde transport_order (filtradas por **dispatch_plan_id**).

El campo **dispatch_plan_id** puede ser nulo.

Si es nulo, trae la lista de ordendes de transporte, si no es nulo solo trae las ordenes de transporte de la planficacion.

C. loop [por cada orden] => arma el detalle de cada fila:

- findRouteId() 11.3** => trae la ruta desde routes (route_id).
- getPlanningTruckId() 11.4** → trae el camión planificado desde planning_truck (route.planning_truck_id).
- findByTruckId() 11.5** => resuelve la placa/tipo desde truck.
- opt [si tiene trip] => findTripId() 11.6** → obtiene el chofer (trip.driver_employee_id).

Response Formato JSON:

```
{
  "status": 200,
  "success": true,
  "data": {
    "dispatchPlanId": 1,  // 0 NULL
    "total": 2,
    "orders": [
      {
        "id": 502,
        "status": "DISPATCHED",
        "route": {
          "routeId": 11,
          "name": "North-Center Route",
          "totalDistanceM": 9800,
          "totalTimeS": 2900
        },
        "truck": {
          "planningTruckId": 2,
          "truckId": 2,
          "plate": "LPZ-1002",
          "type": "DRY",
          "source": "PLANNED"
        },
        "driver": {
          "employeeId": 900,
          "name": "Juan Pérez"
        },
        "load": {
          "weightKg": 120.00,
          "volumeM3": 0.660,
          "capacityWeightKg": 4000.00,
          "occupancyPercent": 3.0
        },
        "tripId": 55,
        "availableActions": [
          "VIEW_DETAIL"
        ]
      }
    ]
  },
  "currentPage": 1,
  "perPage": 20,
  "totalResults": 170,
  "totalPages": 9
}
```

12 Listar puntos de entregar para unificar ordenes de transporte.

Enpoint.

Tipo: (HTTP) POST/delivery-point-unify

Especificación de DTOs y Funciones

Request Principal

El Frontend envia el ID del camion y la lista de ordenes de transporte.

Tabla de atributos deliveryPointsForUnify

Atributo	Tipo	Oblig.	Descripción / Restricción
truckId	number	Sí	ID (PK) del camión.
transportOrders	array	Si	Lista de ordenes de transporte que se necesita consultar

JSON

```
{
  "truckId": 1,
  "transportOrders": [
    {
      "id": 12,
      "tripId": 3,
      "routeId": 4
    },
    {...}
  ]
}
```

A DeliveryPointsForUnify(deliveryPointsForUnify) 12.2

Función del servicio que recibe el objeto desde la request.

Iteracion.

Por cada orden de transporte se buscar su respectivo delivery point.

B getDeliveryPointByTransportOrder(routeId). 12.3

Se consulta en la tabla route la lista de delivery points.

Parametro de entrada: el Id de la ruta.

Salida:

```
[
  {
    "dispatchDeliveryPoint": 5,
    "deliveryPoint": {
      "lat": -17.26565,
      "lon": 49.2656
    },
    "deliveryWindowStart": "08:00",
    "deliveryWindowEnd": "12:00",
    "priority": 1,
    "ownerId": 789,
    "ownerName": "cliente 1",
    "totalWeightKg": 5600,
    "totalVolumeM3": 1600,
    "totalNeto": 560
  },
  {...}
]
```

C listDispatchDeliveryPoint.

Es la lista consolidada de cada routeId asociado a cada orden de transporte y es la respuesta final.

```
[
  {
    "transportOrderId":15,
    "routeId":3,
    "dispatchDeliveryPoint": 5,
    "deliveryPoint": {
      "lat": -17.26565,
      "lon": 49.2656
    },
    "deliveryWindowStart":"14:00",
    "deliveryWindowEnd":"16:00",
    "priority":1,
    "totalWeightKg":4565,
    "totalVolumeM3":230,
    "totalNeto":99
  },
  {
    "transportOrderId":16,
    "routeId":4,
    "dispatchDeliveryPoint": 5,
    "deliveryPoint": {
      "lat": -17.26565,
      "lon": 49.2656
    },
    "deliveryWindowStart":"08:00",
    "deliveryWindowEnd":"10:00",
    "priority":2,
    "totalWeightKg":5600,
    "totalVolumeM3":1600,
    "totalNeto":560
  }, {...}
]
```

13 Mover puntos de entrega entre rutas

Endpoint: ninguno. El envío al backend ocurre recién en "Reoptimizar"

El cache que se muta — sessionStorage

El "Mover" solo modifica el array forcedAssignments; routes no cambia hasta la próxima reoptimización.

```
// key: "optimization:plan:778899"
{
  "dispatchPlanId": 778899,
  "updatedAt": "2026-07-23T14:32:10Z",
  "forcedAssignments": [], // ← pins manuales; lo que muta el "Mover"
  "routes": [ // ← última respuesta de /optimization-route (no cambia al mover)
    {
      "planningTruckId": 1,
      "name": "Ruta 1",
      "visits": [
        {
          "dispatchDeliveryPointId": 401
        },
        {
          "dispatchDeliveryPointId": 407
        }
      ]
    },
    {
      "planningTruckId": 2,
      "name": "Ruta 2",
      "visits": [
        {
          "dispatchDeliveryPointId": 405
        }
      ]
    }
  ]
}
```

Paso a paso

13.1 Seleccionar herramienta — El planificador activa rect o lazo en la MapToolbar (setActiveTool).

13.2 Remarcar en el mapa — Dibuja el área sobre los pines. SelectionLayer calcula los hits y devuelve selectedDeliveryPointIds.

13.3 onMapSelection(ids) — OrdersMap sube la selección a PlanningView, que abre el diálogo "Mover selección" mostrando cuántos puntos hay y un selector de ruta destino.

13.4 Elegir ruta destino + "Mover" — El planificador elige la ruta B; se resuelve su planningTruckId (targetPlanningTruckId).

13.5 Actualizar pins (loop por punto) — Por cada deliveryPointId seleccionado:

pins[deliveryPointId] = targetPlanningTruckId

Se persiste el mapa en sessionStorage (forcedAssignments).

13.6 Repintar el mapa — La UI recolorea los pines movidos con el color de la ruta destino y redibuja el overlay leyendo los pins del cache. No hay ida al backend.

Ejemplo — mover 2 puntos de la Ruta 1 a la Ruta 2

Entrada de la operación:

```
{
  "selectedDeliveryPointIds": [
    401,
    407
  ],
  "targetPlanningTruckId": 2
}
```

Cache — antes:

```
{
  "dispatchPlanId": 778899,
  "forcedAssignments": []
}
```

Cache — después (forcedAssignments actualizado; routes intacto hasta reoptimizar):

```
{
  "dispatchPlanId": 778899,
  "updatedAt": "2026-07-23T14:35:02Z",
}
```

```
    "forcedAssignments": [  
      {  
        "deliveryPointId": 401,  
        "planningTruckId": 2  
      },  
      {  
        "deliveryPointId": 407,  
        "planningTruckId": 2  
      }  
    ]  
  }  
}
```

Conexión con la optimización (doc 09)

Cuando el planificador pulsa "Reoptimizar", el frontend lee los pins del cache y los manda tal cual como forcedAssignments en el request de 09 optimización de rutas:

```
// POST /optimization-route  
{  
  "dispatchPlanId": 778899,  
  "forcedAssignments": [  
    {  
      "deliveryPointId": 401,  
      "planningTruckId": 2  
    },  
    {  
      "deliveryPointId": 407,  
      "planningTruckId": 2  
    }  
  ]  
}
```


14 Procesar Orden de Transporte

ENDPOINT

Tipo: (HTTP) PATCH/process-transport-order
Request.

processTransportOrder(transportOrderId)

Endpoint expuesto que recibe la petición para iniciar el procesamiento e integración con SAP de una orden de transporte existente.

- **Entrada:** transportOrderId (Number — ID de la orden de transporte en la BD local).
- **Retorno:** Objeto de respuesta con el mensaje de éxito/error y el ID asignado por SAP.

JSON de Ejemplo de Entrada (HTTP PATCH Request)

```
{  
  "transportOrderId": 103  
}
```

findTransportOrder(transportOrderId)

Consulta en la base de datos Transport Orders DB la cabecera, el viaje, la ruta, la información del camión y los pedidos de venta asociados a la orden.

- **Firma / Entrada:** transportOrderId (Number).
- **Retorno:** transportOrderData (Object completo extraído de la BD).

Tabla Descriptiva del Retorno (transportOrderData)

Propiedad	Tipo	Descripción / Propósito
id	Number	Identificador único de la orden de transporte.
status	String	Estado actual de la orden en CRM.
assignedWeightKg	Decimal	Peso total asignado a la orden (kg).
assignedVolumeM3	Decimal	Volumen total asignado a la orden (m3).
trip	Object	Datos del viaje y camión asignado.
trip.truckCode	String	Placa o código del vehículo asignado.
trip.departureDate	Timestamp	Fecha y hora planificada de salida.
salesOrders[]	Array	Lista de órdenes de venta vinculadas.
salesOrders[].saleOrderId	Number	Identificador de la orden de venta comercial.

JSON de Ejemplo de Salida (transportOrderData)

```
{
  "id": 103,
  "status": "PENDING",
  "assignedWeightKg": 9100.00,
  "assignedVolumeM3": 2600.00,
  "trip": {
    "truckCode": "3421-ABC",
    "departureDate": "2026-07-24T08:00:00Z"
  },
  "salesOrders": [
    {
      "saleOrderId": 556
    },
    {
      "saleOrderId": 557
    }
  ]
}
```

formatterDataToSap(transportOrderData)

Función interna del Planning Service que transforma y estructura la información obtenida de la base de datos para preparar las solicitudes hacia Ms Sales y SAP.

- **Firma / Entrada:** transportOrderData (Object).

```
{
  "id": 103,
  "status": "PENDING",
  "assignedWeightKg": 9100.00,
  "assignedVolumeM3": 2600.00,
  "trip": {
    "truckCode": "3421-ABC",
    "departureDate": "2026-07-24T08:00:00Z"
  },
  "salesOrders": [
    {
      "saleOrderId": 556
    },
    {
      "saleOrderId": 557
    }
  ]
}
```

- **Retorno:** Objeto formateado con la lista de IDs de venta para validación y la estructura inicial del payload para SAP.

```
{
```

```
"crmTransportOrderId": "103",
"transportType": "0001",
"originLocation": "ALM-CENTRAL",
"scheduledDepartureTime": "2026-07-24T08:00:00Z",
"truckCode": "3421-ABC",
"items": [
  {
    "necesidad": "80001452"
  },
  {
    "necesidad": "80001453"
  }
]
}
```

createTransportOrderSap(transportOrderSapDto) — Integración SAP zws020

Ejecuta la llamada HTTP asíncrona hacia el servicio **zws_orden_transporte_crear** de SAP enviando la orden consolidada.

- **Firma / Entrada:** transportOrderSapDto (Object).
- **Retorno:** sapResponse (Object enviado por SAP BTP).

Tabla Descriptiva de Entrada (transportOrderSapDto)

Campo CRM	Tipo	Oblig.	Descripción / Propósito
crmTransportOrderId	String	Sí	Número de orden de transporte en CRM (ej. "103").
transportType	String	Sí	Clase de transporte en SAP (ej. "0001").
originLocation	String	Sí	Ubicación o almacén de origen de la carga.
scheduledDepartureTime	String	Sí	Fecha y hora planificada de salida en formato ISO/DATETIME.
truckCode	String	Sí	Placa o código de vehículo registrado en SAP.
items[]	Array	Sí	Arreglo de documentos de necesidad devueltos por Ms Sales.
items[].necesidad	String	Sí	Documento de Necesidad asignado al transporte (ej. "80001452").

JSON de Ejemplo de Entrada (transportOrderSapDto)

```
{
  "crmTransportOrderId": "103",
  "transportType": "0001",
  "originLocation": "ALM-CENTRAL",
  "scheduledDepartureTime": "2026-07-24T08:00:00Z",
  "truckCode": "3421-ABC",
  "items": [
    {
      "necesidad": "80001452"
    },
    {
      "necesidad": "80001453"
    }
  ]
}
```

Tabla Descriptiva de Salida (sapResponse)

Campo SAP	Tipo	Descripción / Propósito
orden_transporte_sap	String	Número de documento de transporte generado en SAP.
status	String	Estado de la transacción en SAP ("CREADO", "PENDIENTE", "ERROR").
mensajes	Array	Detalle de respuestas y mensajes informativos/error de SAP.

JSON de Ejemplo de Salida (sapResponse)

```
{
  "orden_transporte_sap": "10005489",
  "status": "CREADO",
  "mensajes": [
    {
      "type": "S",
      "message": "Orden de Transporte 10005489 creada exitosamente en SAP."
    }
  ]
}
```

6. updateTransportOrder({ transportOrderId, sapId })

Actualiza el registro en la tabla transport_orders de la base de datos local para almacenar la confirmación y el ID asignado por SAP, modificando su estado a procesado.

- **Firma / Entrada:** Objeto con transportOrderId (Number) y sapId (String).
- **Retorno:** boolean (true si el registro fue actualizado exitosamente).

Tabla Descriptiva de Parámetros -> Tabla transport_orders

Propiedad	Tipo de Dato	Descripción / Valor a Actualizar
transportOrderId	Number	Identificador único de la orden local a actualizar (103).
sapId	String	Código asignado por SAP ("10005489").
status	String	Cambia estado local a "DISPATCHED" o "PROCESSED_SAP".

JSON de Ejemplo de Entrada

```
{
  "transportOrderId": 103,
  "sapId": "10005489",
  "status": "PROCESSED_SAP"
}
```

JSON de Respuesta Final al Frontend

```
{
  "success": true,
  "code": 200,
  "message": "Orden de transporte procesada e integrada con SAP exitosamente.",
  "data": {
    "transportOrderId": 103,
    "sapTransportOrderId": "10005489",
    "status": "PROCESSED_SAP"
  }
}
```

15 Optimización de Rutas para ordenes de transporte Unificadas.

Enpoint.

Tipo: (HTTP) POST/optimization-route-unify

Especificación de DTOs y Funciones

Request Principal

El frontend envia el id del camion y su detalle de puntos de entregas.

Tabla Descriptiva del Parámetro de Entrada (optimizationRouteUnifyDto)

Propiedad	Tipo	Oblig.	Descripción / Propósito
truckId	Number	Sí	ID del camión seleccionado para realizar la ruta unificada.
data[]	Array[Object]	Sí	Lista de paradas/órdenes candidatas a unificar.
data[].transportOrderId	Number	Sí	ID de la orden de transporte de origen.
data[].routeId	Number	Sí	ID de la ruta calculada previamente para esa orden.
data[].dispatchDeliveryPoint	Number	Sí	ID del punto de entrega consolidado (dispatch_delivery_points.id).
data[].deliveryPoint	Object	Sí	Objeto con las coordenadas GPS del cliente.
data[].deliveryPoint.lat	Number	Sí	Latitud geográfica del punto de entrega.
data[].deliveryPoint.lon	Number	Sí	Longitud geográfica del punto de entrega.
data[].deliveryWindowStart	String	Sí	Hora de inicio de ventana de atención (ej. "08:00").

data[].deliveryWindowEnd	String	Sí	Hora de fin de ventana de atención (ej. "12:00").
data[].priority	Number	Sí	Nivel de prioridad de entrega del cliente.
data[].totalWeightKg	Number	Sí	Peso acumulado del pedido en Kilogramos.
data[].totalVolumeM3	Number	Sí	Volumen acumulado del pedido en Metros Cúbicos.
data[].totalNeto	Number	Sí	Valor económico neto del pedido.

JSON de Ejemplo de Entrada (optimizationRouteUnifyDto)

```
{
  "truckId": 45,
  "data": [
    {
      "transportOrderId": 15,
      "routeId": 3,
      "dispatchDeliveryPoint": 5,
      "deliveryPoint": {
        "lat": -17.26565,
        "lon": 49.2656
      },
      "deliveryWindowStart": "08:00",
      "deliveryWindowEnd": "12:00",
      "sequence": 1,
      "totalWeightKg": 5600,
      "totalVolumeM3": 1600,
      "totalNeto": 560
    },
    {...}
  ]
}
```

2. **getTruckByIdCache(truckId)** — 15.3 / **getTruckById(truckId)** — 15.4

Función encargada de consultar las especificaciones técnicas y capacidades del camión asignado (primero en Redis Caché, y si no existe, en la BD trucks).

Tabla Descriptiva del Resultado (truckData)

Propiedad	Tipo	Descripción / Propósito
truckId	Number	Identificador único del vehículo.
code	String	Código interno del camión en el sistema.
plate	String	Placa de circulación del camión.
capacityWeightKg	Number	Límite máximo de peso en kg que soporta el vehículo.
capacityVolumeM3	Number	Límite máximo de volumen en m ³ que soporta el vehículo.
distributor	Object	Coordenadas GPS del Centro de Distribución de salida.
distributor.lat	Number	Latitud del punto de salida.
distributor.lon	Number	Longitud del punto de salida.

JSON de Ejemplo de Salida (truckData)

```
{
  "truckId": 45,
  "code": "CAM-VOLVO-08",
  "plate": "3421-ABC",
  "capacityWeightKg": 10000,
  "capacityVolumeM3": 30,
  "distributor": {
    "lat": -17.7833,
    "lon": -63.1821
  }
}
```

3. **formatterUnifyToGoogleRoute({ optimizationRouteUnifyDto, truckData })** — 15.6

Transforma el DTO con los puntos de entrega y la información del camión a la estructura oficial que exige Google Route Optimization API.

Tabla Descriptiva del Objeto Generado (googleRouteOptimizationDto)

Propiedad / Ruta JSON	Tipo de Dato	Descripción / Propósito para Google
populatePolylines	Boolean	Solicita a Google que retorne la polilínea codificada para el mapa.
model.vehicles[]	Array[Object]	Lista con la flota de camiones configurada (en este caso 1 vehículo).
model.vehicles[].label	String	Etiqueta única asignada al vehículo.
model.vehicles[].loadLimits	Object	Límites máximos de capacidad física configurados en Google.
model.shipments[]	Array[Object]	Lista de envíos/visitas a realizar en campo.
model.shipments[].label	String	Etiqueta del shipment (combina ID de orden y punto de entrega).
model.shipments[].loadDemands	Object	Peso y volumen requeridos que consumirá la parada.
model.shipments[].deliveries[]	Array[Object]	Ubicación GPS de entrega y tiempo de permanencia/descarga.

JSON de Ejemplo Generado (googleRouteOptimizationDto)

```
{
  "populatePolylines": true,
  "model": {
    "vehicles": [
      {
        "label": "camion-45",
        "loadLimits": {
          "peso_kg": {
            "maxLoad": "10000"
          },
          "volumen_m3": {
            "maxLoad": "30"
          }
        },
        "startLocation": {"latitude": -17.7833,"longitude": -63.1821},
        "endLocation": {"latitude": -17.7833,"longitude": -63.1821}
      }
    ],
    "shipments": [
      {
        "label": "DP-5",
        "loadDemands": {
          "peso_kg": {
            "amount": "5600"
          },
          "volumen_m3": {
            "amount": "16"
          }
        },
        "deliveries": [
          {
            "arrivalLocation": {
              "latitude": -17.26565,
              "longitude": 49.2656
            },
            "duration": "600s"
          }
        ]
      }
    ]
  }
}
```

4. getRoutesGoogle(googleRouteOptimizationDto) — 9.6 / 15.7

Función reutilizable encargada de invocar el endpoint externo de Google Route Optimization y retorna la ruta recalculada.

Tabla Descriptiva de la Respuesta (googleRoutes)

Propiedad / Ruta JSON	Tipo de Dato	Descripción / Propósito
routes[]	Array[Object]	Arreglo con la ruta optimizada por Google.
routes[].vehicleLabel	String	Etiqueta del vehículo asignado.
routes[].routePolyline.points	String	Geometría de la ruta codificada (Encoded Polyline).
routes[].metrics.travelDistanceMeters	Number	Distancia total en metros acumulada.
routes[].metrics.travelDuration	String	Tiempo total estimado de viaje (ej. "4500s").
routes[].routeTotalCost	Number	Costo estimado de la ruta calculado por el motor.
routes[].visits[]	Array[Object]	Secuencia ordenada de visitas óptima.

JSON de Ejemplo de Respuesta (googleRoutes)

```
{
  "routes": [
    {
      "vehicleLabel": "camion-45-3421-ABC",
      "routePolyline": {
        "points": "`lpkBhjs`KnEMnBKxAGEwECwAEwAEo..."
      },
      "metrics": { "travelDistanceMeters": 12560, "travelDuration": "4500s" },
      "routeTotalCost": 120.50,
      "visits": [
        {
          "shipmentLabel": "OT-15_DP-5",
          "startTime": "1970-01-01T08:15:00Z"
        }
      ]
    }
  ]
}
```

5. buildUI(googleRoutes) — 15.7 / 15.8

Función que procesa el resultado de Google y emite la respuesta formateada que la UI utilizará para pintar los pines, la polilínea y los totales en el Frontend.

Tabla Descriptiva de Salida UI (listRoutes)

Propiedad	Tipo de Dato	Descripción / Propósito para la UI
truckId	Number	Identificador único del camión evaluado.
totalCost	Number	Costo operativo monetario total estimado para esta ruta unificada.
etaTotalDistanceM	Number	Distancia total en metros a renderizar en la cabecera/tarjeta del vehículo.
etaTotalTimeS	Number	Tiempo total estimado de tránsito (en segundos).
totalWeightKg	Number	Sumatoria total del peso consolidado que llevará el camión en la ruta.
totalVolumeM3	Number	Sumatoria total del volumen consolidado que ocupará la carga.
encodedPolyline	String	Polilínea codificada para trazar la geometría de la ruta en el mapa.
points	Array[Object]	Lista secuencial y ordenada de las paradas óptimas calculadas.
points.sequence	Number	Orden o número de secuencia de la parada en el trayecto (1, 2, 3...).
points.dispatchDeliveryPoint	Number	Identificador único del punto de entrega (dispatch_delivery_points.id).
points.originalRouteId	Number	ID de la ruta previa a la cual pertenecía este punto de entrega antes de unificar.
points.transportOrderId	Number	ID de la orden de transporte previa asociada a este punto de entrega.
points.eta	String	Hora estimada de arribo (ETA) calculada por Google para esta parada.
points.deliveryPoint	Object	Objeto con las coordenadas geográficas finales (lat, lon).
points.deliveryPoint.lat	Number	Latitud del cliente.
points.deliveryPoint.lon	Number	Longitud del cliente.
points.totalWeightKg	Number	Peso específico que se descargará en esta parada en Kilogramos.

points.totalVolumeM3	Number	Volumen específico que se descargará en esta parada en metros cubicos
Points.totalNeto	Number	Monto o valor económico asociado a la entrega de este cliente.

JSON de Ejemplo de Salida UI (listRoutes)

```
{
  "truckId": 45,
  "totalCost": 120.50,
  "etaTotalDistanceM": 12560,
  "etaTotalTimeS": 4500,
  "totalWeightKg": 9100,
  "totalVolumeM3": 2600,
  "encodedPolyline": "`lpkBhjs`KnEMnBKxAGEwECwAEwAEo...",
  "points": [
    {
      "sequence": 1,
      "dispatchDeliveryPoint": 5,
      "originalRouteId": 3,
      "originalTripId": 3,
      "transportOrderId": 15,
      "code": 150,
      "eta": "08:15:00",
      "deliveryPoint": {
        "lat": -17.26565,
        "lon": 49.2656
      },
      "totalWeightKg": 5600,
      "totalVolumeM3": 1600,
      "totalNeto": 560
    },
    {
      "sequence": 2,
      "dispatchDeliveryPoint": 8,
      "originalRouteId": 4,
      "originalTripId": 3,
      "transportOrderId": 18,
      "eta": "09:30:00",
      "deliveryPoint": {
        "lat": -17.28110,
        "lon": 49.2890
      },
      "totalWeightKg": 3500,
      "totalVolumeM3": 1000,
      "totalNeto": 420
    }
  ]
}
```


16 CREACION DE ORDENES DE TRANSPORTE FINALIZADA.

ENDPOINT

Tipo: (HTTP) POST/create-transport-order

Especificaciones de Dtos y funciones.

1. createTransportOrder(unifyTransportOrderDto) — 16.1 / 16.2

Procesa la petición de guardado definitivo tras la unificación, coordinando la anulación de entidades previas y el registro de la nueva Orden de transporte.

Tabla Descriptiva de Entrada (unifyTransportOrderDto)

Propiedad	Tipo de Dato	Oblig.	Descripción / Propósito
truckId	Number	Sí	Identificador del camión asignado al nuevo viaje.
driverId	Number	Sí	Identificador del chofer/conductor responsable.
helperId	Number	No	Identificador del ayudante/asistente (puede ser null).
totalCost	Number	Sí	Costo monetario estimado recalculado para la ruta.
etaTotalDistanceM	Number	Sí	Distancia total en metros de la ruta unificada.
etaTotalTimeS	Number	Sí	Tiempo total en segundos de la ruta unificada.

totalWeightKg	Number	Sí	Peso total acumulado de la carga (\$kg\$).
totalVolumeM3	Number	Sí	Volumen total acumulado de la carga (\$m^3\$).
encodedPolyline	String	Sí	Geometría codificada de la nueva ruta unificada.
points	Array[Object]	Sí	Lista de paradas/puntos de entrega ordenados.
points.sequence	Number	Sí	Secuencia o número de parada (1, 2, 3...).
points.dispatchDeliveryPoint	Number	Sí	ID del punto de entrega (dispatch_delivery_points.id).
points.originalRouteId	Number	Sí	ID de la ruta previa a anular/desvincular.
points.originalTripId	Number	Sí	ID del viaje previo a anular/desvincular.
points.transportOrderId	Number	Sí	ID de la orden de transporte previa a anular.
points.code	Number	No	Código de referencia previo de la orden.
points.eta	String	Sí	Hora estimada de llegada al punto.
points.deliveryPoint	Object	Sí	Coordenadas GPS (lat, lon) del cliente.
points.totalWeightKg	Number	Sí	Peso específico a entregar en esta parada.
points.totalVolumeM3	Number	Sí	Volumen específico a entregar en esta parada.
points.totalNeto	Number	Sí	Monto económico neto de los productos.

JSON.

```
{
  "truckId": 45,
  "driverId": 78,
  "helperId": null,
  "totalCost": 120.50,
  "etaTotalDistanceM": 12560,
  "etaTotalTimeS": 4500,
  "totalWeightKg": 9100,
  "totalVolumeM3": 2600,
  "encodedPolyline": "`lpkBhjs`KnEMnBKxAGEwECwAEwAEo...",
  "points": [
    {
      "sequence": 1,
      "dispatchDeliveryPoint": 5,
      "originalRouteId": 3,
      "originalTripId": 3,
      "transportOrderId": 15,
      "code": 150,
      "eta": "08:15:00",
      "deliveryPoint": {
        "lat": -17.26565,
        "lon": 49.2656
      },
      "totalWeightKg": 5600,
      "totalVolumeM3": 1600,
      "totalNeto": 560
    },
    {...}
  ]
}
```

Fase de Creación: Especificación de Funciones y Mapeo con BD

createTransportOrder(transportOrderDto) — Inserción en Tabla transport_orders y transport_order_sales 16.9

Registra la nueva **Orden de Transporte** que agrupa el peso/volumen total y reasigna los enlaces comerciales hacia las órdenes de venta originales en transport_order_sales.

Tabla Descriptiva (transportOrderDto) -> Tabla transport_orders y transport_order_sales

Propiedad	Tipo	Oblig.	Descripción / Propósito
dispatchPlanId	Null	No	Setea en null por tratarse de una mezcla de planes.
tripId	Number	Sí	FK hacia la tabla trips (trip_id: 50).
routeId	Number	Sí	FK hacia la tabla routes (route_id: 9999).
status	String	Sí	Estado inicial de ejecución ("PENDING" o "LOADING").
checkedBy	Null	No	Usuario revisor de rampa (inicia en null).
assignedWeightKg	Decimal	Sí	Sumatoria total del peso de la carga
assignedVolumeM3	Decimal	Sí	Sumatoria total del volumen de la carga
salesOrderIds[]	Array[Number]	Sí	Lista de IDs de Órdenes de Venta (sales_order_id) heredadas de las órdenes consolidadas.

JSON de Entrada (transportOrderDto)

```
{
  "dispatchPlanId": null,
  "tripId": 50,
  "routeId": 9999,
  "status": "PENDING",
  "checkedBy": null,
  "assignedWeightKg": 9100,
  "assignedVolumeM3": 2600,
  "createdBy": "gino.gonzales",
  "salesOrderIds": [
    556,
    557,
    558
  ]
}
```

- **Retorna:** transportOrderId (Number — ID de la nueva Orden de Transporte, ej. 103)

createTransportOrderSap(transportOrderSapDto) — Integración con SAP (zws020) 16.11

Se encarga de estructurar el payload que se enviará al iFlow de SAP Integration Suite a partir de la orden maestra recién creada.

Tabla Descriptiva (transportOrderSapDto) -> Servicio zws_orden_transporte_crear

Propiedad	Tipo	Oblig.	Descripción / Propósito
crmTransportOrderId	String	Sí	ID de la Orden creada en CRM ("103").
transportType	String	Sí	Clase de transporte configurada en SAP (ej. "0001").
originLocation	String	Sí	Código de centro/almacén de despacho.
scheduledDepartureTime	Timestamp	Sí	Fecha/hora programada del viaje (departure_date).
truckPlate	String	Sí	Placa del camión para validación contra zws_camion_listar.
items[]	Array	Sí	Documentos de necesidad/entrega asociados.
items[].necesidad	String	Sí	Código del documento de necesidad en SAP.// array de id del documento necesario de SAP

JSON de Entrada (transportOrderSapDto)

```
{
  "crmTransportOrderId": "103",
  "transportType": "0001",
  "originLocation": "ALM-CENTRAL",
  "scheduledDepartureTime": "2026-07-24T08:00:00Z",
  "truckPlate": "3421-ABC",
  "items": [
    { "necesidad": "80001452" },
    { "necesidad": "80001453" }
  ]
}
```

Retorno de la Función

- **Retorna:** sapResponse (Object con status: "CREADO", sapTransportOrderId: "10005489").

5. updateTransportOrder({transportOrderId,sapId}) — 16.12.

Funcion que se encarga de actualizar el Id de SAP en el campo code de la tabla Transport Orders,

Json

```
{
  "transportOrderId": 456,
  "sapId": 100004567 // en el campo code
}
```

17 Obtener Camiones y rutas

Endpoint: GET /trucks?distributorId={id}
Tipo: HTTP GET
Obtener la lista de camiones y las ordenes de transporte asignadas.

17.1 Request (Frontend → Gateway Controller → Planning Controller)
Especificación de DTO: FilterTrucksDto

Atributo	Tipo	Obligatorio	Descripción
distributorId	number	Sí	ID de la distribuidora asociada al planificador en sesión. Scope del listado.
date	string (ISO/Date)	No	Filtro por fecha de salida programada
tripStatus	string	No	Filtro por estado de los trips
plate	string	No	Filtro parcial por placa del vehículo.
truckType	string	No	Filtro por tipo de vehículo (DRY, FRIO).

Formato JSON:

```
{
  "distributorId": 1,
  "date": "2026-07-24",
  "status": "PENDING",
  "plate": null,
  "truckType": null
}
```

17.2 Flujo del Proceso Backend

17.3 transport_orders - findByDistributorId(distributorId)

Recupera los viajes de la distribuidora.

Formato JSON:

```
"trips": [  
  {  
    "id": 40,  
    "truckId": 56,  
    "distributorId": 1,  
    "driverEmployeeId": null,  
    "helperEmployeeId": null,  
    "status": "PENDING",  
    "departureDate": "2026-07-24T08:00:00Z"  
  },  
  {  
    "id": 41,  
    "truckId": 57,  
    "distributorId": 1,  
    "driverEmployeeId": 900,  
    "helperEmployeeId": 901,  
    "status": "PENDING",  
    "departureDate": "2026-07-24T08:00:00Z"  
  }  
]
```

17.4 (LOOP): trips - findById(trip.truckId)

Obtener camiones

Formato JSON:

```
{  
  "id": 56,  
  "plate": "3421-ABC",  
  "capacityWeightKg": 28000,  
  "capacityVolumeM3": 62.0,  
  "truckType": "DRY"  
}
```

17.5 routes - findByTripId(trip.id, { orderStatus })

Obtenemos las rutas al TRIP

Formato JSON:


```
[
  {
    "id": 2051,
    "tripId": 40,
    "dispatchPlanId": 778899,
    "planningTruck": {
      "id": 759,
      "assignedWeightKg": 4556,
      "assignedVolumeM3": 1520
    },
    "isSeleted": true
  },
  {
    "id": 2052,
    "tripId": 40,
    "dispatchPlanId": 778899,
    "planningTruckId": 789,
    "planningTruck": {
      "id": 757,
      "assignedWeightKg": 3000,
      "assignedVolumeM3": 700
    },
    "isSeleted": true
  }
]
```

Respuesta del Endpoint (Response)

```
{
  "status": 200,
  "success": true,
  "data": {
    "distributorId": 1,
    "trips": [
      {
        "tripId": 40,
        "departureDate": "2026-07-24T08:00:00Z",
        "tripStatus": "PENDING",
        "truck": {
          "truckId": 56,
          "plate": "3421-ABC",
          "type": "DRY",
          "capacityWeightKg": 28000.00,
          "capacityVolumeM3": 62.00
        },
        "routescount": [
          {
            "id": 2051,
            "tripId": 40,
            "dispatchPlanId": 778899,
            "planningTruck": {
              "id": 759,
              "assignedWeightKg": 4556,
              "assignedVolumeM3": 1520
            },
            "isSeleted": true
          },
          {...}
        ],
        "load": {
          "weightKg": 29400.00,
          "volumeM3": 87.00,
          "occupancyPercent": 105.0
        }
      },
      {...}
    ]
  }
}
```

19 Obtener ordenes Despachadas

Endpoint.

Tipo: (HTTP) GET /monitoring/orders

Obtener el estado actual de todas las órdenes de transporte despachadas de una distribuidora, con su camión, su progreso de entregas y la última posición conocida del camión.

Especificación de DTOs y funciones.

Request Principal (GET /monitoring/orders?{distributorId})

Se requiere el Id de la distribuidora.

Parámetros de entrada: filterMonitoringDto

Tabla de atributos filterMonitoringDto

Atributo	Tipo	Oblig.	Descripción / Restricción
distributorId	number	Sí	ID de la distribuidora.
plate	string	No	Placa del camión
tripStatus	string	No	Estado del viaje: PENDING, EN_RUTA, FINALIZADO.

```
"filterMonitoringDto": {  
  "distributorId": 1,  
  "plate": "3456", // Opcional  
  "transportStatus": "EN_RUTA", // Opcional  
}
```

GetMonitoringOrders(distributorId) 19.2 y 19.3

El **Gateway Controller** recibe el **GET /monitoring/orders (19.1)** y lo delega al **Monitoring Controller (19.2)** que valida la estructura de los query params y llama al **Monitoring Service (19.3)**.

- Parametro de entrada: **filterMonitoringDto**

FindDispatchedOrders(distributorId) 19.4

Método para obtener las ordenes de transporte despachadas de la distribuidora desde la tabla Transport Order. Una fila del listado es una orden de transporte

Tabla de atributos de listTransportOrder

Atributo	Tipo TypeScript	Oblig.	Descripción / Restricción
transportOrderId	number	Sí	transport_order.id
distributorId	number	Sí	transport_order.distributor_id
routeId	number null	No	transport_order.route_id
orderStatus	string	Sí	transport_order.status
assignedWeightKg	number	Sí	transport_order.assigned_weight_kg
assignedVolumeM3	number	Sí	transport_order.assigned_volume_m3

Ejemplo JSON:

```
[
  {
    "transportOrderId": 4471,
    "distributorId": 1, // Id de la distribuidora
    "routeId": 512, // Id de la ruta
    "orderStatus": "DISPATCHED", // Estado de la orden
    "assignedWeightKg": 3480.50,
    "assignedVolumeM3": 14.20
  },
  { "...": "..." }
]
```

getRoutesByIds(routeIds) 19.6 y join trucks 19.6a

Método responsable de resolver la salida de cada orden: camión, chofer, hora de salida y estado. Se llama una vez con el arreglo de **routeId**.

Tabla de atributos de listRoute:

Atributo	Tipo	Oblig.	Descripción / Restricción
routeId	number	Sí	routes.id
distributorId	number	Sí	routes.distributor_id
truckId	number null	No	routes.truck_id
licensePlate	string	Sí	trucks.plate
driverEmployeeId	number null	No	routes.driver_employee_id
nameDriverEmployee	string	Sí	routes.name_driver_employee
transportStatus	string	Sí	Estado de la salida física
departureDate	string null	No	routes.departure_date
completedDate	string null	No	routes.completed_date

Ejemplo JSON:

```
[
  {
    "routeId": 512,
    "distributorId": 1,
    "truckId": 880012,
    "licensePlate": "3456-ABC",
    "driverEmployeeId": 456,
    "nameDriverEmployee": "Carlos Mamani",
    "transportStatus": "EN_RUTA",
    "departureDate": "2026-07-16T08:00:00.000Z",
    "completedDate": null
  },
  { "...": "..." }
]
```

CountStatusesByOrder(orderIds) 19.7

Método responsable de calcular, los 3 contadores que la tabla muestra, progreso, paradas e incidencias.

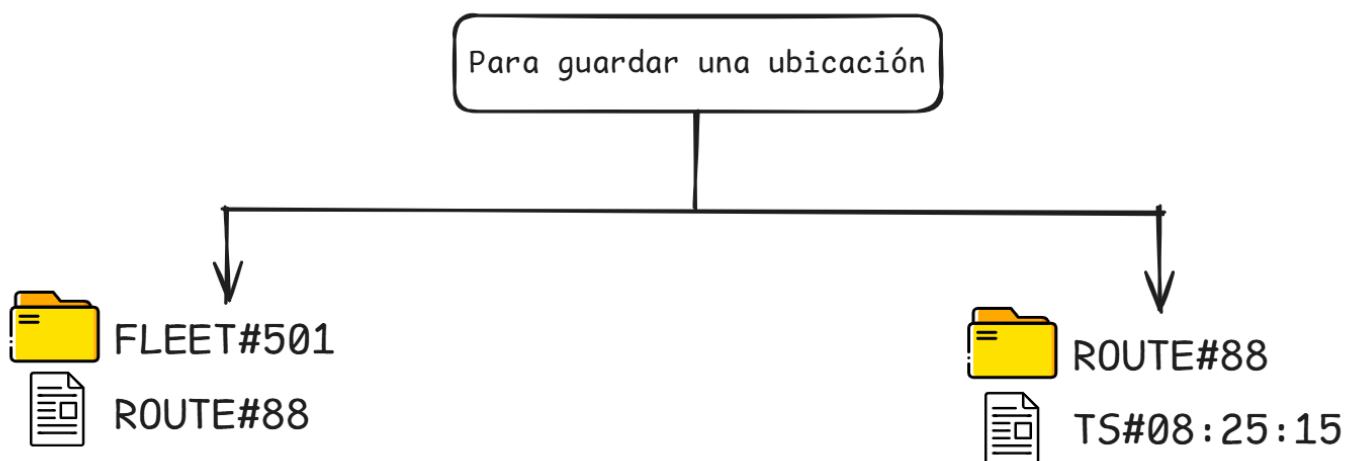
Tabla de atributos de progressDto

Atributo	Tipo TypeScript	Oblig.	Descripción / Restricción
total	number	Sí	count(delivery_orders)
delivered	number	Sí	Entregas con delivery_orders.status de entrega cumplida
failed	number	Sí	Entregas cerradas sin entregar
returned	number	Sí	Entregas cerradas con devolución
pending	number	Sí	Derivado: total - (delivered + failed)
progressPct	number	Sí	Derivado: cerradas / total
incidents	number	Sí	count(delivery_incidents) de las entregas de la orden

Ejemplo JSON:

```
{
  "total": 12,
  "delivered": 6,
  "failed": 1,
  "returned": 0,
  "pending": 5,
  "progressPct": 58,
  "incidents": 1,
  "outOfWindow": 0
}
```

Query DynamoDB: 19.8



query(PK=FLEET#{distributorId})

Método responsable de traer la última posición conocida de toda la flota de la distribuidora desde DynamoDB.

Query TableName = truck_tracking

KeyConditionExpression = "pk = :pk"

ExpressionAttributeValues = { ":pk": "FLEET#1" }

Una sola Query para todas las ordenes.

Tabla de atributos de listItemActual:

Atributo	Tipo TypeScript	Oblig.	Descripción / Restricción
pk	string	Sí	<u>FLEET</u> #{distributorId}. Clave de partición.
sk	string	Sí	<u>ORD</u> #{transportOrderId}. Clave de ordenamiento.
latitude	number	Sí	Grados decimales, 6 decimales
longitude	number	Sí	Grados decimales, 6 decimales
battery	number	Sí	Batería del dispositivo del chofer.
trackedAt	string	Sí	Reloj del DISPOSITIVO: cuándo el GPS fijó la posición.
receivedAt	string	Sí	Reloj del SERVIDOR: cuándo llegó el paquete.

Ejemplo JSON:

```
[
  {
    "pk": "FLEET#501",
    "sk": "ROUTE#88",
    "latitude": -17.783412,
    "longitude": -63.181245,
    "battery": 74,
    "trackedAt": "2026-07-16T08:24:39.000Z",
    "receivedAt": "2026-07-16T08:24:40.180Z"
  },
  { "...": "..." }
]
```


MergeTrackingByTripId(orders, items) 19.9

Metodo responsable de cruzar las filas de postgres con los items de DynamoDB.

Tabla de atributos de MonitoringOrderDto

Atributo	Tipo TypeScript	Oblig.	Descripción / Restricción
transportOrderId	number	Sí	transport_order.id
code	string	Sí	Código visible de la orden
tripId	number null	No	transport_order.trip_id
routeId	number null	No	transport_order.route_id
orderStatus	string	Sí	transport_order.status
truckPlate	string	Sí	trucks.plate
driverName	string	Sí	trips.name_driver_employee
transportStatus	string	Sí	trips.status
departureDate	string null	No	trips.departure_date
progress	progressDto	Sí	Contadores
tracking	trackingDto null	No	Ítem ACTUAL

Sub-DTO: progressDto

```
{
  "total": 12,
  "delivered": 6,
  "failed": 1,
  "returned": 0,
  "pending": 5,
  "progressPct": 58,
  "incidents": 1,
  "outOfWindow": 0
}
```

Sub-DTO: trackingDto

Atributo	Tipo TypeScript	Oblig.	Descripción / Restricción
routeId	number	Sí	Resuelto desde sk (ROUTE#{tripId})
latitude	number	Sí	DynamoDB truck_tracking.latitude
longitude	number	Sí	DynamoDB truck_tracking.longitude
battery	number	Sí	DynamoDB truck_tracking.battery, 0-100
trackedAt	string	Sí	DynamoDB truck_tracking.trackedAt
receivedAt	string	Sí	DynamoDB truck_tracking.receivedAt

Ejemplo de respuesta a frontend formato JSON:

```
{
  "success": true,
  "code": 200,
  "data": [
    {
      "transportOrderId": 4471,
      "code": "OT-2026-004471",
      "tripId": 88,
      "routeId": 512,
      "orderStatus": "DISPATCHED",
      "truckPlate": "3456-ABC",
      "driverName": "Carlos Mamani",
      "tripStatus": "EN_RUTA",
      "departureDate": "2026-07-16T08:00:00.000Z",
      "progress": {
        "total": 12,
        "delivered": 6,
        "failed": 1,
        "pending": 5,
        "progressPct": 58,
        "incidents": 1,
      },
      "tracking": {
        "tripId": 88,
        "latitude": -17.783412,
        "longitude": -63.181245,
        "battery": 74,
        "trackedAt": "2026-07-16T08:24:39.000Z",
        "receivedAt": "2026-07-16T08:24:40.180Z"
      }
    },
    { "...": "..." }
  ]
}
```

Suscripción al stream (SSE) 19.10 Endpoint.

Tipo: (HTTP) GET /monitoring/stream?distributorId={id}

Mantener el listado actualizado enviando solo lo que cambió, sin reenviar nunca la flota entera.

Pasos de la secuencia.

- **GET** /monitoring/stream?distributorId={id} **19.10**
 - Se realiza una sola vez al abrir la pantalla.
 - Una sola conexion con SCOPE de FLOTA
- **openStream(distributorId)** **19.11**
- **subscribe(FLEET#{distributorId})** **19.12**

24 Tracking del camión

Endpoint.

Tipo: (HTTP) POST /monitoring/tracks

Registrar un ping de posición del camión. Lo envía la app del chofer, automáticamente y sin acción del usuario.

POST /monitoring/tracks (24.1)

Ejemplo JSON (payload de tracking): trackDto

```
{
  "routeId": 512,
  "distributorId": 1,
  "latitude": -17.783412,
  "longitude": -63.181245,
  "employeeId": 456,
  "trackedAt": "2026-07-16T08:24:39.000Z",
  "battery": 0.74
}
```

stampReceivedAt(track) 24.4

Método responsable de estampar el reloj del servidor.

Ejemplo JSON con el receivedAt estampado.

```
{
  "routeId": 512,
  "distributorId": 1,
  "latitude": -17.783412,
  "longitude": -63.181245,
  "employeeId": 456,
  "trackedAt": "2026-07-16T08:24:39.000Z",
  "receivedAt": "2026-07-16T08:24:40.180Z",
  "battery": 74
}
```

putItem(TRAZA) 24.5

Método responsable de escribir el ítem la TRAZA en **truck_tracking**. Se escribe un ítem por ping, nunca se pisa. Responde la pregunta "¿por dónde anduvo ESTA salida?".

```
PK  ROUTE#{routeId}
SK  TS#{trackedAt}
```

- última posición de la salida -> Query PK, ScanIndexForward=false, Limit=1
- recorrido entre dos horas -> Query PK, SK BETWEEN 'TS#t1' AND 'TS#t2'

Ejemplo JSON (ítem TRAZA escrito)

```
{
  "pk": "ROUTE#512",
```

```

"sk": "TS#2026-07-16T08:24:39.000Z",
"latitude": -17.783412,
"longitude": -63.181245,
"battery": 74,
"employeeId": 456,
"receivedAt": "2026-07-16T08:24:40.180Z",
"expiresAt": 1786782279
}

```

putItem(Actual) 24.5

Método responsable de escribir el ítem ACTUAL. Es overwrite: un ítem por salida en curso, y cada ping pisa el anterior. Existe solo para que la flota entera de una distribuidora salga en una Query.

```

PK FLEET#{distributorId}      (misma forma, otro origen)
SK ROUTE#{routeId}

```

Ejemplo JSON (ítem ACTUAL escrito)

```

{
  "pk": "FLEET#1",
  "sk": "ROUTE#512",
  "latitude": -17.783412,
  "longitude": -63.181245,
  "battery": 74,
  "trackedAt": "2026-07-16T08:24:39.000Z",
  "receivedAt": "2026-07-16T08:24:40.180Z"
}

```

publish(tracking, routeId) 24.7

Método responsable de publicar el evento **tracking** en el bus **SSE** en memoria, con **SCOPE** de salida.

Ejemplo JSON (evento tracking en cualquiera de los dos streams)

```

{
  "routeId": 512,
  "latitude": -17.783412,
  "longitude": -63.181245,
  "battery": 74,
  "trackedAt": "2026-07-16T08:24:39.000Z",
  "receivedAt": "2026-07-16T08:24:40.180Z"
}

```

patchByRouteId(item) 24.10

Método del cliente, al recibir el evento, parchea su estado local buscando por **routeId** y deja intactas las demás filas o paradas. Nunca se reemplaza la colección entera.

```
{
  ...prev, // La data actual renderizada.
  [id]: { newData } // El elemento nuevo, actualizado. Se renderiza solo 1 columna.
}
```

Response principal formato JSON:

Devuelve la confirmación de que el **PING** se aceptó y se escribió. No devuelve el item: El cliente que lo mando ya lo tiene, y la app del chofer no consume el estado del monitor.

```
{
  "success": true,
  "code": 202,
  "message": "Ping registrado.",
  "data": {
    "routeId": 512,
    "trackedAt": "2026-07-16T08:24:39.000Z",
    "receivedAt": "2026-07-16T08:24:40.180Z"
  }
}
```

25 Detalle

Endpoint.

Tipo: (HTTP) POST /monitoring/tracks

Registrar un ping de posición del camión. Lo envía la app del chofer, automáticamente y sin acción del usuario.

POST /monitoring/tracks (24.1)

Ejemplo JSON (payload de tracking): trackDto

18 Revision a Ciegas

Endpoint.

Tipo: (HTTP) GET /blind-count/transport-orders

Obtener la lista de las ordenes de transporte para realizar el conteo a ciegas

Especificación de DTOs y funciones.

Tabla Descriptiva del Query Parameter (Entrada)

Propiedad	Tipo de Dato	Oblig.	Descripción / Propósito
driverId	Number	Sí	Identificador del empleado chofer/despachador autenticado (driver_employee_id).
statusFilter	String	No	Filtro de estado para la lista (ALL, PENDING, CHECKED). Por defecto ALL.

JSON de Ejemplo de Entrada (HTTP Query Parameters)

```
{
  "driverId": 78,
  "statusFilter": "ALL"
}
```

getActiveTripsByDriver(driverId) — 18.3 / 18.4

Consulta a la base de datos Trips DB para obtener los viajes activos (status IN ('PENDING', 'EN_RUTA')) asignados al chofer.

- **Entrada:** driverId (Number).
- **Retorno:** Lista de viajes activos en la tabla trips.

Tabla Descriptiva del Retorno (activeTripsList)

Propiedad	Tipo de Dato	Campo equivalente en BD	Descripción / Propósito
id	Number	trips.id	Identificador único del viaje físico.
truckId	Number	trips.truck_id	Identificador del camión asignado.
driverEmployeeId	Number	trips.driver_employee_id	Identificador del chofer.
status	String	trips.status	Estado operativo del viaje (PENDING, EN_RUTA).

JSON de Ejemplo de Salida (activeTripsList)

```
[
  {
    "id": 50,
    "truckId": 45,
    "driverEmployeeId": 78,
    "status": "PENDING"
  }
]
```

getTransportOrdersByTripIds(tripIds, statusFilter) — 18.5 / 18.6

Consulta las órdenes de transporte registradas en transport_orders asociadas a los viajes obtenidos.

- **Entrada:** tripIds (Array[Number]), statusFilter (String).
- **Retorno:** Lista de cabeceras de órdenes de transporte.

Tabla Descriptiva del Retorno (transportOrdersList)

Propiedad	Tipo	Descripción / Propósito
id	Number	Identificador único de la orden de transporte.
code	Number	Código correlativo de la OT (ej. 1000456).
tripId	Number	FK del viaje al que pertenece.
routeId	Number	FK de la ruta optimizada asociada.
status	String	Estado del conteo (PENDING, CHECKED_OK, DISCREPANCY, APPROVED).
assignedWeightKg	Decimal	Peso total de la orden en Kilogramos.
assignedVolumeM3	Decimal	Volumen total de la orden en m3.
etaTotalTimeS	Decimal	Duración total en segundos estimada para la ruta.
ownerName	String	Nombre de la distribuidora o cliente principal.

JSON de Ejemplo de Salida (transportOrdersList)

```
[
  {
    "id": 15,
    "code": 1000456,
    "tripId": 50,
    "routeId": 9999,
    "status": "PENDING",
    "assignedWeightKg": 120.00,
    "assignedVolumeM3": 15.00,
    "etaTotalTimeS": 25200,
    "ownerName": "Distribuidora Andina"
  },
  {
    "id": 18,
    "code": 1000458,
    "tripId": 50,
    "routeId": 10002,
    "status": "CHECKED_OK",
    "assignedWeightKg": 120.00,
    "assignedVolumeM3": 15.00,
    "etaTotalTimeS": 25200,
    "ownerName": "Mayorista Central"
  }
]
```

countDeliveryStopsByRouteIds(routeIds) — 18.7 / 18.8

Consulta a route_delivery_points para contar cuántas paradas activas (is_active = true) componen la ruta de cada orden de transporte.

- **Entrada:** routeIds (Array[Number]).
- **Retorno:** Mapa/Objeto asociativo con el conteo de paradas por route_id.

JSON de Ejemplo de Salida (stopsCount)

```
{
  "9999": 12,
  "10002": 12
}
```

buildBlindCountListResponse(transportOrdersList, stopsCount) — 18.9 / 18.10

Función interna del servicio que formatea el DTO final consumido por la App Móvil, calculando dinámicamente los contadores de la barra superior (*Todos, Pendientes, Cargados*).

- **Entrada:** transportOrdersList (Array), stopsCount(Object).
- **Retorno:** blindCountOrdersResponse (Object).

Tabla Descriptiva del Retorno Final (blindCountOrdersResponse)

Propiedad	Tipo de Dato	Descripción / Propósito para la App Móvil
summary	Object	Objeto contenedor con los totales para las pestañas/badges del header.
summary.totalOrders	Number	Total de órdenes de transporte asignadas al chofer.
summary.pendingOrders	Number	Total de órdenes en estado pendiente de conteo (PENDING).
summary.checkedOrders	Number	Total de órdenes finalizadas o cargadas (CHECKED_OK, APPROVED).
orders	Array[Object]	Lista formateada de las tarjetas de órdenes de transporte.
orders.transportOrderId	Number	ID único de la orden de transporte local (transport_orders.id).
orders.otCode	String	Formato visual del código de OT (ej. "OT-1000456").
orders.ownerName	String	Nombre del cliente/distribuidora asociada a la orden.
orders.distributorId	Number	Identificador de la distribuidora.
orders.stopsCount	Number	Cantidad de paradas de entrega que tiene la orden.
orders.estimatedHours	Number	Tiempo estimado redondeado en horas (calculado a partir de eta_total_time_s).
orders.totalWeightKg	Number	Peso total de la orden en kg.
orders.status	String	Estado operativo traducido para la App (PENDING, CHECKED_OK, DISCREPANCY, APPROVED).

orders.statusLabel	String	Texto de la etiqueta visual ("Pendiente", "Cargado", "Aprobado").
--------------------	--------	---

JSON de Ejemplo de Salida Final (bLindCountOrdersResponse)

```
{
  "summary": {
    "totalOrders": 6,
    "pendingOrders": 2,
    "checkedOrders": 4
  },
  "orders": [
    {
      "transportOrderId": 15,
      "otCode": "OT-1000456",
      "ownerName": "Distribuidora Andina",
      "distributorId": 2045,
      "stopsCount": 12,
      "estimatedHours": 7,
      "totalWeightKg": 120,
      "status": "PENDING",
      "statusLabel": "Pendiente"
    },
    {
      "transportOrderId": 18,
      "otCode": "OT-1000458",
      "ownerName": "Mayorista Central",
      "distributorId": 2044,
      "stopsCount": 12,
      "estimatedHours": 7,
      "totalWeightKg": 120,
      "status": "CHECKED_OK",
      "statusLabel": "Cargado"
    }
  ]
}
```

21 Obtener detalle de una orden de transporte (conteo a ciegas).

Endpoint.

Tipo: (HTTP) GET /blind-count/transport-orders

Obtener el detalle de la orden de transporte que el usuario selecciono.

Especificación de DTOs y funciones.

getBlindCountOrderDetails(transportOrderId) — 21.1 / 21.2

Tabla Descriptiva de Entrada (transportOrderId)

Propiedad	Tipo de Dato	Oblig.	Descripción / Propósito
transportOrderId	Number	Sí	Identificador único de la orden de transporte en la BD local (transport_orders.id).

JSON / Path Parameter de Entrada.

```
{  
  "transportOrderId": 15  
}
```

findTransportOrder(transportOrderId) — 21.3 / 21.4

Consulta las métricas generales y estado actual de la Orden de Transporte en la base de datos TransportOrders DB.

Tabla Descriptiva de Entrada

Propiedad	Tipo de Dato	Oblig.	Campo Equivalente en BD	Descripción
transportOrderId	Number	Sí	transport_orders.id	ID de la orden a buscar.

JSON de Ejemplo de Entrada

```
{  
  "transportOrderId": 15  
}
```

Tabla Descriptiva de Salida (transportOrderData)

Propiedad	Tipo de Dato	Campo Equivalente en BD	Descripción
id	Number	transport_orders.id	Identificador único de la orden.
code	Number	transport_orders.code	Código visual de la orden en CRM/SAP.
status	String	transport_orders.status	Estado operativo en base de datos.

JSON de Ejemplo de Salida (transportOrderHeader)

```
{  
  "id": 15,  
  "code": 1000456,  
  "status": "COUNTING"  
}
```

getOrderItems(transportOrderId) — 21.5 / 21.6

Obtiene los ítems de venta asociados a la orden y los cruza con el catálogo de (product_snapshot) para recuperar el factor de conversión y banderas de cadena de frío.

Tabla Descriptiva de Entrada

Propiedad	Tipo	Oblig.	Campo Equivalente en BD	Descripción
transportOrderId	Number	Sí	transport_order_sales.transport_order_id	ID de la orden para consultar sus ítems.

JSON de Ejemplo de Entrada

```
{
  "transportOrderId": 15
}
```

Tabla Descriptiva de Salida (requestedItemsList)

Propiedad	Tipo de Dato	Descripción / Propósito
productId	Number	Identificador único del producto.
productCode	String	Código SKU o EAN de barra.
productName	String	Nombre comercial del producto.
isRefrigerated	Boolean	Marca de producto frío .
unitsPerBox	Integer	Factor de conversión de cajas a unidades.
expectedQty	Decimal	Cantidad total programada en Unidades mínimas .

JSON de Ejemplo de Salida (requestedItemsList)

```
[
  {
    "productId": 30201,
    "productCode": "7790001",
    "productName": "Ketchup 900ml",
    "isRefrigerated": false,
    "unitsPerBox": 12,
    "expectedQty": 24
  },
  {
    "productId": 30204,
    "productCode": "7790004",
    "productName": "Levadura Fleischmann 1Kg",
    "isRefrigerated": true,
    "unitsPerBox": 10,
    "expectedQty": 20
  }
]
```

getExistingCountsByTransportOrderId(transportOrderId) — 21.7 / 21.8

Consulta los datos guardados en truck_inventories (inventory) para saber qué productos y cantidades en unidad mínima ya registró el chofer previamente.

Tabla Descriptiva de Entrada

Propiedad	Tipo de Dato	Oblig	Campo Equivalente en BD	Descripción
transportOrderId	Numero	Sí	truck_inventories.transport_order_id	ID de la orden de transporte a consultar.

Tabla Descriptiva de Salida (existingCountsList)

Propiedad	Tipo de Dato	Descripción
productId	Number	Identificador del producto guardado.
loadedQty	Decimal	Cantidad contada acumulada en Unidades mínimas .
expectedQty	Decimal	Cantidad esperada registrada al crear el ítem.
varianceQty	Decimal	Diferencia (loaded_qty - expected_qty).

JSON de Ejemplo de Salida (existingCountsList)

```
[
  {
    "productId": 30201,
    "loadedQty": 1,
    "expectedQty": 24,
    "varianceQty": -23
  },
  {
    "productId": 30204,
    "loadedQty": 20,
    "expectedQty": 20,
    "varianceQty": 0
  }
]
```

applyBlindCount(requestedItemsList, existingCountsList) — 21.9

Función de lógica de negocio pura que fusiona las dos listas, oculta expectedQty si el producto no es frío, calcula banderas de diferencia (hasVariance) y construye el objeto de respuesta UI.

Tabla Descriptiva de Entradas

Propiedad	Tipo de Dato	Oblig.	Descripción / Origen
requestedItemsList	Array[Object]	Sí	Resultado devuelto por getOrderItemsWithMetadata().
existingCountsList	Array[Object]	Sí	Resultado devuelto por getExistingCountsByTransportOrderId().

JSON de Ejemplo de Entrada

```
{
  "requestedItemsList": [
    {
      "productId": 30201,
      "productCode": "7790001",
      "productName": "Ketchup 900ml",
      "isRefrigerated": false,
      "unitsPerBox": 12,
      "expectedQty": 24
    }
  ],
  "existingCountsList": [
    {
      "productId": 30201,
      "loadedQty": 1,
      "expectedQty": 24,
      "varianceQty": -23
    }
  ]
}
```

Tabla Descriptiva de Salida (blindCountOrderDetailResponse)

Propiedad	Tipo de Dato	Descripción / Regla Aplicada
summary	Object	Objeto con métricas resumidas (okCount, diffCount).
countedItems[]	Array[Object]	Lista procesada con enmascaramiento aplicado.
countedItems[].hasVariance	Boolean	true si varianceQty != 0.
countedItems[].varianceStatus	String	"MATCH" si varianceQty == 0, "MISMATCH" no coinciden.

JSON de Ejemplo de Salida

```
{
  "transportOrderId": 15,
  "otCode": "1000456",
  "status": "COUNTING",
  "summary": {
    "okCount": 1,
    "diffCount": 1
  },
  "countedItems": [
    {
      "productId": 30201,
      "productCode": "7790001",
      "productName": "Ketchup 900ml",
      "isRefrigerated": false,
      "unitsPerBox": 12,
      "loadedQtyMin": 1,
      "hasVariance": true,
      "varianceStatus": "MISMATCH"
    }
  ]
}
```

22 Registro y eliminacion de Items.

Fase de Registro

Endpoint para registrar un Item.

Tipo: (HTTP) POST /blind-count/transport-orders/:itemId/items

Registrar un producto con su respectivo conteo

Especificación de DTOs y funciones.

registerCountItem(transportOrderId, registerCountItemDto) — 22.1 / 22.2

Endpoint invocado al presionar el botón "Registrar Ítem" en la App Móvil para guardar o actualizar el conteo de un producto.

Tabla Descriptiva del Parámetro de Entrada (registerCountItemDto)

Propiedad	Tipo	Oblig.	Descripción / Propósito
transportOrderId	Number	Sí	ID de la orden de transporte enviado en la URL (transport_orders.id).
productId	Number	Sí	Identificador único del producto contado (product_snapshot.product_id).
quantityMin	Decimal	Sí	Cantidad en unidad mínima enviada por la App Móvil (UNIDAD).
createdBy	String	Sí	Usuario/empleador que realiza el registro (ej. "driver_78").

JSON de Ejemplo de Entrada (registerCountItemDto)

```
{
  "productId": 30201,
  "quantityMin": 1,
  "createdBy": "driver_78"
}
```

Tabla Descriptiva del Datos de Salida (registerItemResponse)

Propiedad	Tipo	Descripción / Propósito para la UI
success	Boolean	Confirma que el registro fue procesado con éxito.
item	Object	Objeto con la información del ítem actualizado.
item.productId	Number	Identificador del producto.
item.productCode	String	Código/SKU para mostrar en la tarjeta (ej. "7790001").
item.productName	String	Nombre del producto.
item.loadedQtyMin	Decimal	Cantidad total acumulada en Unidades mínimas .
item.hasVariance	Boolean	true si existe diferencia entre lo contado y lo esperado.
item.varianceStatus	String	Estado visual ("MISMATCH", "MATCH").
summary	Object	Totales actualizados para la cabecera superior (okCount, diffCount).

JSON de Ejemplo de Salida (registerItemResponse)

```
{
  "success": true,
  "item": {
    "productId": 30201,
    "productCode": "7790001",
    "productName": "Ketchup 900ml",
    "isRefrigerated": false,
    "unitsPerBox": 12,
    "loadedQtyMin": 1,
    "hasVariance": true,
    "varianceStatus": "MISMATCH"
  },
  "summary": {
    "okCount": 1,
    "diffCount": 1
  }
}
```

getExpectedQuantityByProduct(transportOrderId, productId) — 22.3 / 22.4

Consulta en la base de datos TransportOrderSales DB (transport_order_sale_items) la cantidad originalmente solicitada/programada para ese producto en la orden.

Tabla Descriptiva del Parámetro de Entrada

Propiedad	Tipo de Dato	Oblig.	Campo Equivalente en BD	Descripción
transportOrderId	Number	Sí	transport_order_sales.transport_order_id	ID de la orden de transporte.
productId	Number	Sí	transport_order_sale_items.product_id	ID del producto a verificar.

JSON de Ejemplo de Entrada

```
{
  "transportOrderId": 15,
  "productId": 30201
}
```

Tabla Descriptiva de Datos de Salida (expectedQtyResponse)

Propiedad	Tipo de Dato	Campo Equivalente en BD	Descripción
productId	Number	transport_order_sale_items.product_id	ID del producto.
expectedQty	Decimal	transport_order_sale_items.quantity_min	Sumatoria esperada en Unidades mínimas .

JSON de Ejemplo de Salida (expectedQtyResponse)

```
{
  "productId": 30201,
  "expectedQty": 24
}
```

upsertTruckInventory(inventoryDto) — 22.5 / 22.6

Guarda o actualiza la fila en la tabla truck_inventories (inventory), registrando la cantidad contada, la esperada y la diferencia calculada.

Tabla Descriptiva del Parámetro de Entrada (inventoryDto) -> Tabla truck_inventories

Propiedad	Tipo de Dato	Oblig.	Descripción / Propósito
truckId	Number	Sí	ID del camión asignado a la orden.
transportOrderId	Number	Sí	FK de la orden de transporte.
productId	Number	Sí	ID del producto.
loadedQty	Decimal	Sí	Cantidad física ingresada por el chofer (\$1\$).
expectedQty	Decimal	Sí	Cantidad oficial esperada (\$24\$).
varianceQty	Decimal	Sí	Diferencia (\$1 - 24 = -23\$).
status	String	Sí	Estado del registro ("MISMATCH" o "MATCH").
updatedBy	String	Sí	Usuario que realiza el cambio.

JSON de Ejemplo de Entrada (inventoryDto)

```
{
  "truckId": 45,
  "transportOrderId": 15,
  "productId": 30201,
  "loadedQty": 1,
  "expectedQty": 24,
  "varianceQty": -23,
  "status": "MISMATCH",
  "updatedBy": "driver_78"
}
```


Tabla Descriptiva de Datos de Salida (upsertInventoryResponse)

Propiedad	Tipo de Dato	Campo en BD (truck_inventories)	Descripción
inventoryId	Number	truck_inventories.id	Identificador autogenerado de la fila.
status	String	truck_inventories.status	Estado guardado en base de datos.

JSON de Ejemplo de Salida (upsertInventoryResponse)

```
{
  "inventoryId": 8042,
  "status": "MISMATCH"
}
```

Fase de Anulacion

Endpoint para anular el conteo de un Item.

Tipo: (HTTP) DELETE /blind-count/transport-orders/:itemId/items

removeCountItem(transportOrderId, productId) — 22.8 / 22.9

Endpoint invocado al presionar el icono del basurero en la App Móvil para revertir/eliminar el conteo de un producto específico.

Tabla Descriptiva de Parámetros de Entrada (Path Variables)

Propiedad	Tipo de Dato	Oblig.	Descripción / Propósito
transportOrderId	Number	Sí	ID de la orden de transporte en la URL (transport_orders.id).
productId	Number	Sí	ID del producto a eliminar en la URL (product_snapshot.product_id).

JSON / Path Parameters de Entrada

```
{
  "transportOrderId": 15,
  "productId": 30201
}
```

Tabla Descriptiva de Datos de Salida (removeItemResponse)

Propiedad	Tipo de Dato	Descripción / Propósito para la UI
success	Boolean	Confirma la eliminación de la fila en el servidor.
removedProductId	Number	ID del producto removido para que la UI lo retire de la lista.
summary	Object	Totales recalculados para las pestañas/badges superiores.

JSON de Ejemplo de Salida (removeItemResponse)

```
{
  "success": true,
  "removedProductId": 30201,
  "summary": {
    "okCount": 1,
    "diffCount": 0
  }
}
```

deleteTruckInventory(transportOrderId, productId) — 22.10 / 22.11

Ejecuta el borrado (o borrado lógico via deleted_at = NOW()) de la fila en la tabla truck_inventories.

Tabla Descriptiva de Parámetros de Entrada -> Tabla truck_inventories

Propiedad	Campo en BD	Tipo de Dato	Oblig.	Descripción
transportOrderId	transport_order_id	Number	Sí	ID de la orden de transporte.
productId	product_id	Number	Sí	ID del producto a borrar.

JSON de Ejemplo de Entrada

```
{  
  "transportOrderId": 15,  
  "productId": 30201  
}
```

Tabla Descriptiva de Datos de Salida (deleteInventoryResponse)

Propiedad	Tipo	Descripción
affectedRows	Number	Número de filas eliminadas/actualizadas en la BD

JSON de Ejemplo de Salida (deleteInventoryResponse)

```
{  
  "affectedRows": 1  
}
```

23 Finalizacion de Revision a ciegas.

Endpoint para registrar un Item.

Tipo: (HTTP) POST /blind-count/transport-orders/:transportOrderId/finalize

Al tocar el botón "Finalizar" se procesa el cierre de la revision a ciegas de la Orden de Transporte.

Especificación de DTOs y funciones.

finalizeBlindCount(transportOrderId, finalizeBlindCountDto) — 23.1 / 23.2

Tabla Descriptiva del Parámetro de Entrada (finalizeBlindCountDto)

Propiedad	Tipo de Dato	Oblig.	Descripción / Propósito
transportOrderId	Number	Sí	ID de la orden de transporte enviado en el Path (transport_orders.id).
checkedBy	String	Sí	Identificador/Usuario del chofer o despachador que finaliza el conteo.
notes	String	No	Observaciones opcionales ingresadas durante el proceso.

JSON de Ejemplo de Entrada (finalizeBlindCountDto)

```
{
  "checkedBy": "driver_78",
  "notes": "Carga revisada"
}
```

getInventoryVariancesByTransportOrderId(transportOrderId) — 23.3 / 23.4

Consulta todas las filas registradas para esta orden de transporte en la tabla truck_inventories (inventory) para evaluar las variaciones.

Tabla Descriptiva del Parámetro de Entrada

Propiedad	Tipo	Oblig.	Campo en BD	Descripción / Propósito para la UI
transportOrderId	Number	Sí	truck_inventories.transport_order_id	ID de la orden de transporte a evaluar.

JSON de Ejemplo de Entrada

```
{
  "transportOrderId": 15
}
```

Tabla Descriptiva de Datos de Salida (inventoryVariancesList)

Propiedad	Tipo	Descripción / Propósito para la UI
inventoryId	Number	Identificador de la fila en inventario.
productId	Number	Identificador del producto.
loadedQty	Decimal	Cantidad contada en unidades mínimas.
expectedQty	Decimal	Cantidad esperada en unidades mínimas.
varianceQty	Decimal	Diferencia calculada).
status	String	Estado del ítem ("MATCH" o "MISMATCH").

JSON de Ejemplo de Salida (inventoryVariancesList)

```
[
  {
    "inventoryId": 8042,
    "productId": 30201,
    "loadedQty": 1,
    "expectedQty": 24,
    "varianceQty": -23,
    "status": "MISMATCH"
  },
  {
    "inventoryId": 8043,
```

```

"productId": 30204,
"loadedQty": 20,
"expectedQty": 20,
"varianceQty": 0,
"status": "MATCH"
}
]

```

updateTransportOrderStatus(transportOrderId, updateStatusDto) — 23.6 / 23.8

Actualiza la cabecera de la tabla transport_orders (exec_order) registrando el nuevo estado del conteo y la firma del usuario responsable.

Tabla Descriptiva del Parámetro de Entrada (updateStatusDto) -> Tabla transport_orders

Propiedad	Campo (transport_orders)	Tipo de Dato	Oblig.	Descripción / Propósito
transportOrderId	id	Number	Sí	Identificador de la orden de transporte a actualizar.
status	status	String	Sí	Estado asignado ("CHECKED_OK" o "DISCREPANCY").
checkedBy	checked_by	String	Sí	Usuario que firmó/completó la revisión física en rampa.
updatedBy	updated_by	String	Sí	Identificador para la auditoría de modificación.

JSON de Ejemplo de Entrada (updateStatusDto)

```

{
  "transportOrderId": 15,
  "status": "DISCREPANCY",
  "checkedBy": "driver_78",
  "updatedBy": "driver_78"
}

```

Tabla Descriptiva de Datos de Salida

Propiedad	Tipo de Dato	Descripción
affectedRows	Number	Cantidad de registros actualizados en la BD.

JSON de Ejemplo de Salida

```

{

```

```
"affectedRows": 1
}
```

createTransportOrderHistory(historyDto) — 23.7 / 23.9

Inserta un registro de trazabilidad en la tabla transport_order_histories (transp_ord_hist) para auditoría del proceso.

Tabla Descriptiva del Parámetro de Entrada (historyDto) -> Tabla transport_order_histories

Propiedad	Campo en BD (transport_order_histories)	Tipo	Oblig.	Descripción / Propósito
transportOrderId	transport_order_id	Number	Sí	FK de la orden de transporte.
status	status	String	Sí	Estado transicionado ("CHECKED_OK" o "DISCREPANCY").
description	description	Text	Sí	Detalle informativo sobre el cierre del conteo a ciegas.
createdBy	created_by	String	Sí	Usuario que generó el evento de auditoría.

JSON de Ejemplo de Entrada (historyDto)

```
{
  "transportOrderId": 15,
  "status": "DISCREPANCY",
  "description": "Conteo a ciegas finalizado con diferencias, pendiente de revision de supervisor",
  "createdBy": "driver_78"
}
```

Tabla Descriptiva de Datos de Salida

Propiedad	Tipo de Dato	Campo en BD	Descripción
historyId	Number	transport_order_histories.id	ID autogenerado del evento de historial.

JSON de Ejemplo de Salida

```
{
  "historyId": 5012
}
```

FinalizeBlindCountResponse 23.10

Tabla descriptiva.

Propiedad	Tipo de Dato	Descripción / Propósito para la UI
success	Boolean	Confirma la correcta finalización del proceso.
transportOrderId	Number	Identificador de la orden procesada.
otCode	String	Código correlativo visual de la OT (ej. "OT-1000456").
finalStatus	String	Estado final asignado a la orden ("CHECKED_OK" o "DISCREPANCY").
statusLabel	String	Etiqueta en texto para el modal de la App ("Cargado" o "Con Diferencias").
hasDiscrepancies	Boolean	true si la orden registró descuadre en algún ítem.
summary	Object	Resumen cuantitativo final del conteo.
summary.totalItemsCounted	Number	Total de ítems registrados en el inventario.
summary.okCount	Number	Cantidad de productos cuya cantidad coincidió exactamente.
summary.discrepancyCount	Number	Cantidad de productos que presentaron diferencias.
message	String	Mensaje descriptivo para la notificación en pantalla.

JSON de Ejemplo de Salida (finalizeBlindCountResponse)

```
{
  "success": true,
  "transportOrderId": 15,
  "otCode": "OT-1000456",
  "finalStatus": "DISCREPANCY",
  "statusLabel": "Con Diferencias",
  "hasDiscrepancies": true,
  "summary": {
    "totalItemsCounted": 2,
    "okCount": 1,
    "discrepancyCount": 1
  },
  "message": "Conteo finalizado con 1 diferencia detectada. La orden queda marcada para revisión del supervisor."
}
```